NeuroStack

Agentic AI Architecture and Design

Overview: NeuroStack is a general-purpose *agentic AI* platform that enables enterprises to build multi-agent systems with reasoning, memory, and orchestration at scale. Our developed system leverage base level frameworks like Crew AI and provides multiple layers of services and features that enable agentic flows to be implemented within enterprise environments in a safe, scalable and robust manner. It adopts an **eight-layer architecture** that spans from core infrastructure up to operations/governance. Each layer's role and implementation are defined below, with logical and deployment architecture diagrams, and Azure- native components as primary. Where relevant, alternative implementations using Kong Gateway or Portkey AI Gateway are noted to provide enterprise clients alternatives to achieve their business and technical requirements. We then detail the **AI Gateway** (enterprise control point) design, enhancements to **Cognition & Reasoning** (continuous learning via RL), **Memory & Personalization** (long- term memory and user-specific context), cross-industry **use case architectures**, and **deployment strategies** (SaaS vs. private cloud). Overall Neurostack is designed to provide an enterprise ready full stack agentic development for multiple use cases and lays the foundation for enterprise clients to fully leverage agentic flows.

1. Eight-Layer Agentic AI Architecture

To ground the design, NeuroStack follows the 8-layer agentic AI framework, ensuring each concern is modular and scalable. **Figure 1** below illustrates these layers and key components:

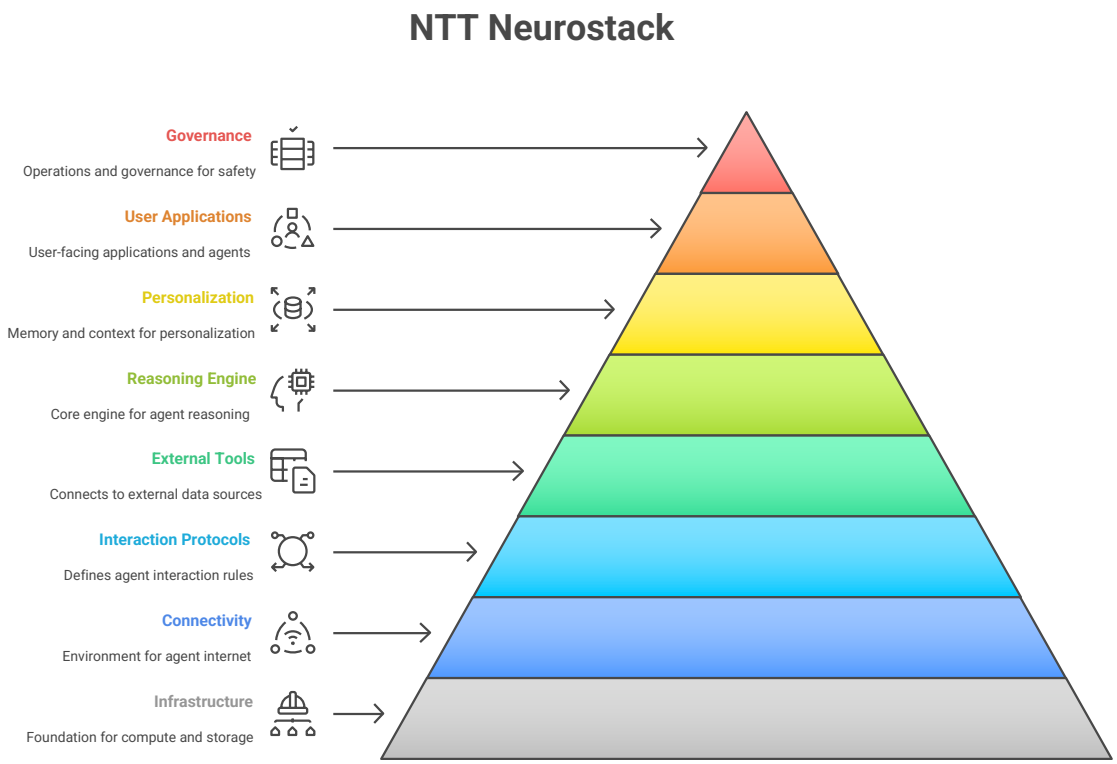
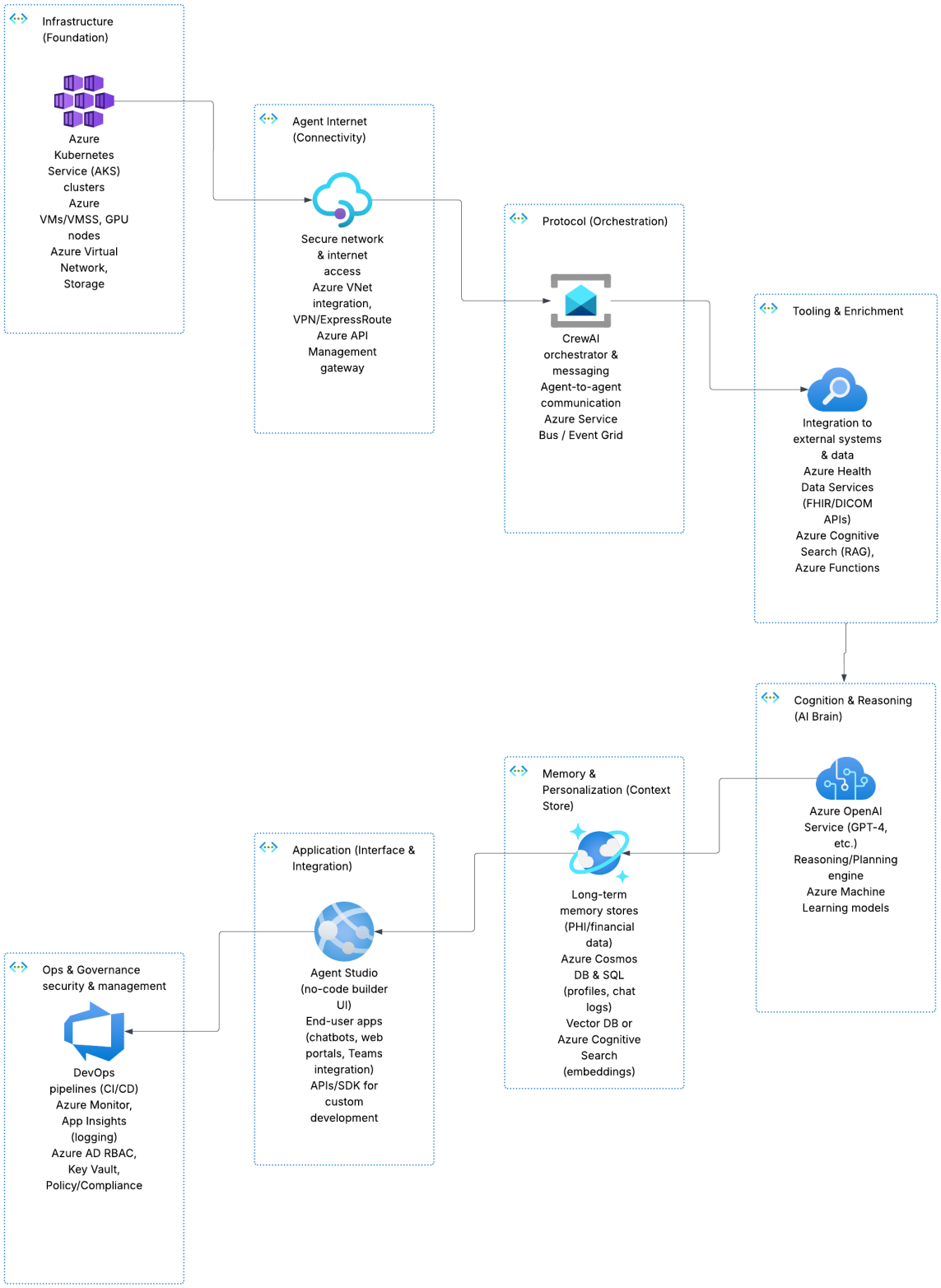


Figure 1: Eight-Layer Agentic AI Architecture

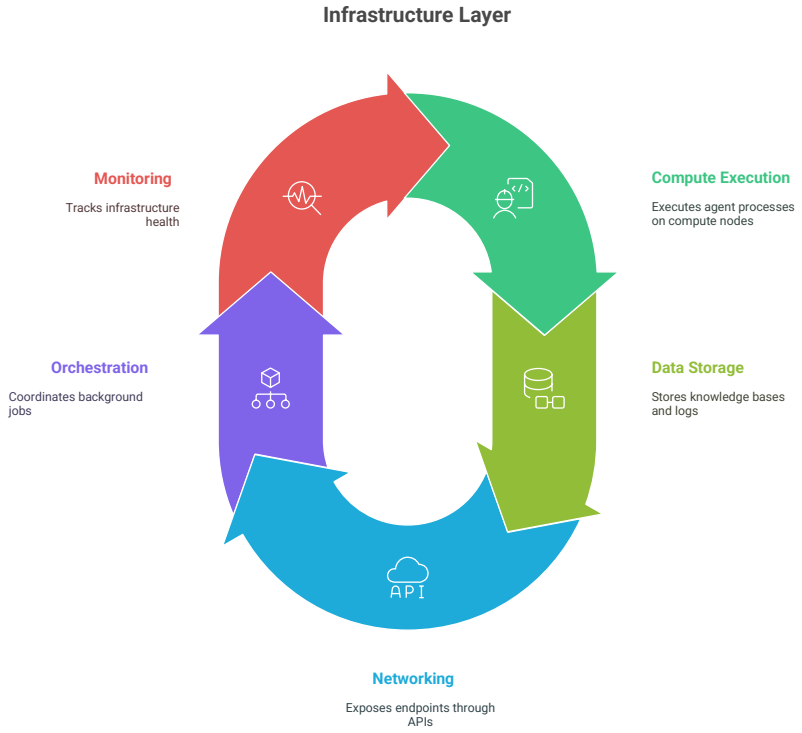
Each layer is defined and designed as follows:



Layer 1: Infrastructure Layer (Foundation)

Role & Logic: The Infrastructure layer comprises the foundational cloud and network elements that all higher layers rely on. It includes computing resources, networking, storage, and base services to support reliable operations. Logically, this layer provides *scalable compute* (e.g. containers, VMs, serverless), *storage* for data and models, *APIs and networking* (e.g. HTTP/REST endpoints, WebSockets), and *orchestration* systems for workflows. All other layers are built on this stable foundation.

Logical Architecture: At this layer, the platform ensures high availability and performance through cloud infrastructure. For example, a logical view might include a **compute cluster** (for running agents and tools), a **data lake/storage** for datasets and logs, a **networking** subsystem for internal and external connectivity (load balancers, API gateways, CDN), and **orchestration** services for scheduling tasks (e.g. Airflow or Azure Data Factory for data pipelines). These components work in concert: the compute nodes execute agent processes; data stores hold knowledge bases, prompts, and logs; networking/APIs expose endpoints; and orchestrators coordinate background jobs. Monitoring hooks are present at this layer to track system health.

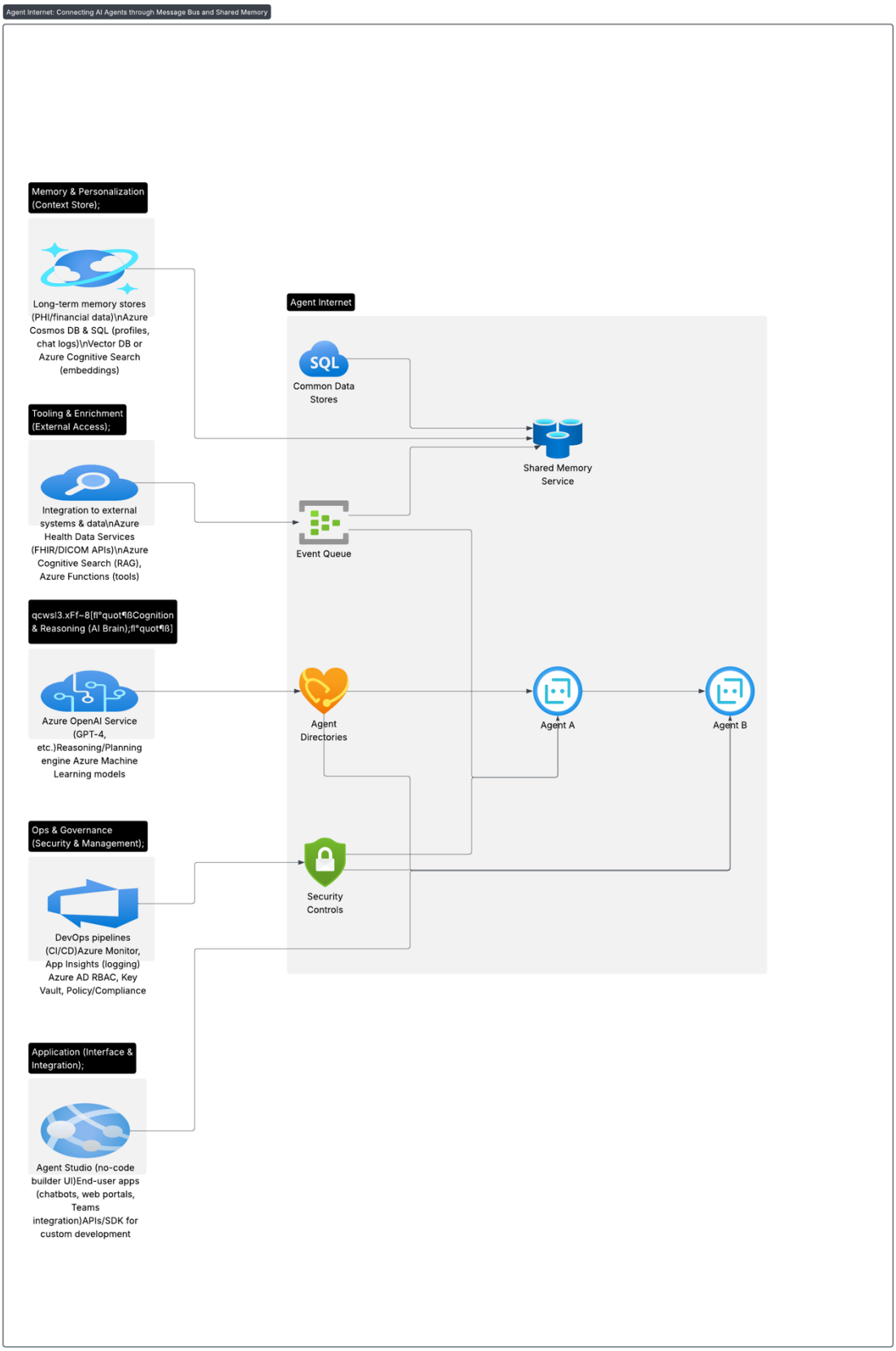


Deployment Architecture: In Azure, this layer is implemented with cloud-native services. Compute can be provided by **Azure Kubernetes Service (AKS)** clusters (to deploy multiple agent services and tools), along with **Azure VM Scale Sets** or **Azure Functions** for serverless tasks. Data storage uses **Azure Blob Storage/ Data Lake** for large files and datasets, and **Azure SQL/Managed Instance** or **Azure Cosmos DB** for structured data and metadata. Networking and APIs are handled by **Azure Virtual Network** (for internal microservices), **Azure Front Door or CDN** (for global routing and caching static content), and **Azure Application Gateway** or **Load Balancers** for traffic distribution. **Azure API Management (APIM)** serves as an entry point for client requests (as part of the Gateway, see Layer 2). Orchestration and scheduling can use **Azure Logic Apps** or **Durable Functions** for event-driven flows, and Azure's managed workflow services for data pipelines. Monitoring is enabled via **Azure Monitor** and **Application Insights** for logs/

metrics. In summary, the Azure deployment ensures that core infrastructure – compute, storage, network – can scale to enterprise workloads and meet reliability/security requirements.

Variations: The infrastructure layer can be hosted in private cloud or on-prem if needed. For instance, on-premises deployments might use Kubernetes on VMs (with NFS storage and a self-managed database cluster) to mimic the above setup. The key is to provide an equivalent foundation of compute, networking, and storage.

Layer 2: Agent Internet Layer (Communication & Coordination)



5

Role & Logic: The Agent Internet layer enables communication and coordination among distributed agents and system components. It effectively acts as the "interconnect" or network that agents use to find each other, share data, and collaborate. This includes protocols for agent messaging, broadcast channels for events, and shared data networks (e.g. a common knowledge hub or memory space accessible to all agents). The layer supports distributed agent topologies such as **agent meshes** or **societies** of agents, where information and tasks flow between agents. It also encompasses *agent identity and state management* across the network, and might provide discovery services so agents can dynamically locate tools or other agents.

Logical Architecture: Logically, this layer can be visualized as a **message bus or agent mesh network** connecting multiple agents. Agents publish and subscribe to events or tasks on this "agent internet." For example, Agent A might emit an event that Agent B listens to when a task is completed. A **shared memory service** may exist so that agents have a common " bulletin board" or knowledge repository (e.g. a vector database or memory cache) to read/write information accessible by others. Key components include: (1) a **communication medium** (such as an event queue, pub/sub system, or P2P messaging) through which agents exchange messages asynchronously; (2) *agent directories or registries* to register agent endpoints and capabilities; (3) *common data stores* (like an enterprise knowledge graph or vector index) that any agent can query as a form of an "intranet of knowledge"; and (4) security controls to ensure agents only access permitted info. The "Agent Internet" name suggests that just as the human internet connects services, this connects AI agents and resources in a network-like fashion. For instance, **embedding stores** (vector DBs like Pinecone or Weaviate) can be part of this layer, serving as a shared semantic memory accessible over the network by all agents. Also, *agent-to-agent dialogues* can occur via this layer using standardized channels.

Deployment Architecture: In Azure, the agent internet is realized with cloud messaging and networking services. **Azure Service Bus or Event Grid** can broadcast events (e.g. "PatientSummaryCompleted" event after an agent finishes summarization) that other agents subscribe to. This decoupling via events allows reliable multi-agent workflows: if an agent fails, the message can be retried or picked up elsewhere, boosting resilience. For direct agent-to-agent calls, **internal APIs** are exposed – e.g. agents running on AKS might have REST/gRPC endpoints for specific functions. These can be published through an internal APIM or via Kubernetes service discovery, enforcing a common interface for calls. All such calls adhere to a standard schema or protocol (JSON payloads or gRPC contracts) so that even custom-coded agents follow the same communication rules. An example: a "Manager" agent might call a "Worker" agent's `performTask` API with a JSON payload containing the task details, expecting a standardized response.

For shared knowledge, **Azure Cognitive Search with vector index** can serve as a central knowledge hub accessible by all agents via secure API calls. This means any agent can query a shared search index (which contains company-wide data or collective memory), effectively creating an *agent knowledge intranet*. Alternatively, an **Azure Cosmos DB** or **Redis** cache can act as a shared blackboard for agents to post intermediate results or read each other's outputs. Each agent is assigned an identity (with credentials managed via Azure AD or an internal registry) so that communications can be authenticated/authorized. The network layer also includes **ACLs and VNet rules** to isolate agent communication in a secure environment.

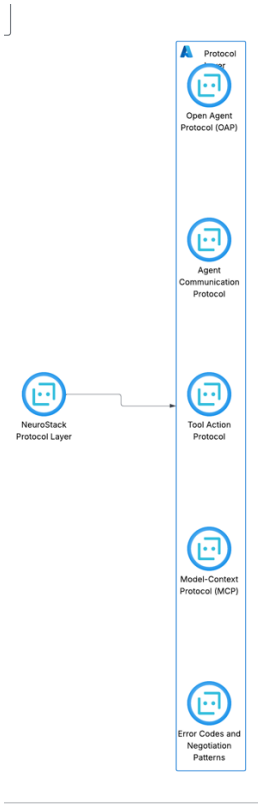
Variations: If using **Kong Gateway** or **Portkey** for agent communication: Kong can facilitate internal service discovery through its Service/Route model – each agent's API can be a Kong service, so agents can call `http://kong/agentX/api` to reach Agent X. Kong's mesh or Gateway can handle authentication and routing. Portkey, while primarily an AI gateway for model calls, could theoretically be extended for agent-to-agent function calls if agents expose functions in a standardized way (Portkey supports remote tool calls via MCP, which could include agent endpoints). In summary, the Agent Internet can be implemented via any robust messaging fabric or API gateway that connects multiple agent processes, as long as it supports the needed protocols and security.

Layer 3: Protocol Layer (Standards & Interoperability)

Role & Logic: The Protocol layer defines **standard communication methods and schemas** that govern how agents interact, how they invoke tools, and how tasks are described across the platform. Essentially, it is the "language" of the agent ecosystem – ensuring that all agents and tools speak a compatible protocol. This covers *Agent-to-Agent (A2A) protocols*, *Agent-to-Tool calling conventions*, Model Context Protocol, task delegation formats, error handling conventions, and any negotiation or handshaking needed. By having a clear protocol layer, the system achieves interoperability between heterogeneous agents and tools, and can integrate with external agent frameworks as well.

Logical Architecture: Logically, this layer provides **specifications and middleware**. For example, an **Agent Communication Protocol** might define how an agent expresses a task request to another agent (e.g. JSON with fields: `agent_id`, `task`, `payload`, etc.) and how the response is structured. A *Tool Action Protocol*

defines how an agent calls external functions (for instance, formatting a tool invocation as a JSON-RPC or OpenAI function call object). In emerging standards, this includes protocols like **Open Agent Protocol (OAP)** – a community effort to standardize cross-agent communication and capability sharing – and **Model-Context Protocol (MCP)** – an open protocol that standardizes how tools and context are provided to LLMs. The protocol layer in NeuroStack embraces such standards so that agents can operate in a larger ecosystem. It also includes things like **Tool/Action Policy (TAP)** or Agent Gateway Protocols that formalize how agents request tool usage and get results. Additionally, error codes and negotiation patterns are part of this layer (e.g. an agent returns a specific error message if it lacks info, prompting another agent or a human to supply missing data).



Deployment Architecture: Implementation-wise, much of the Protocol layer is in software (within the CrewAI/NeuroStack framework). For example, the platform might provide a **Python SDK or library** that all agents use, which ensures they send messages in the correct format and parse responses accordingly. If an agent's function is exposed via Azure APIM or Functions, the *APIM policy* can enforce that requests conform to a JSON schema (validating protocol compliance). Within the code, CrewAI's framework likely already enforces a structure for tasks and tool calls. For instance, CrewAI's agent calls to tools are decorated and follow LangChain's `@tool` conventions, which essentially is a protocol for tool usage. NeuroStack wraps each external tool in a *standard Azure Function microservice* with a uniform REST interface (input JSON with `tool_name`, `parameters`, etc., and output JSON result). This means when an agent wants to invoke a tool, it doesn't call it arbitrarily – it sends a well-formed request to the tool service. The orchestrator (Layer 4/5) mediates these calls, so the agent isn't executing code directly but making a service call, which is logged and secured.

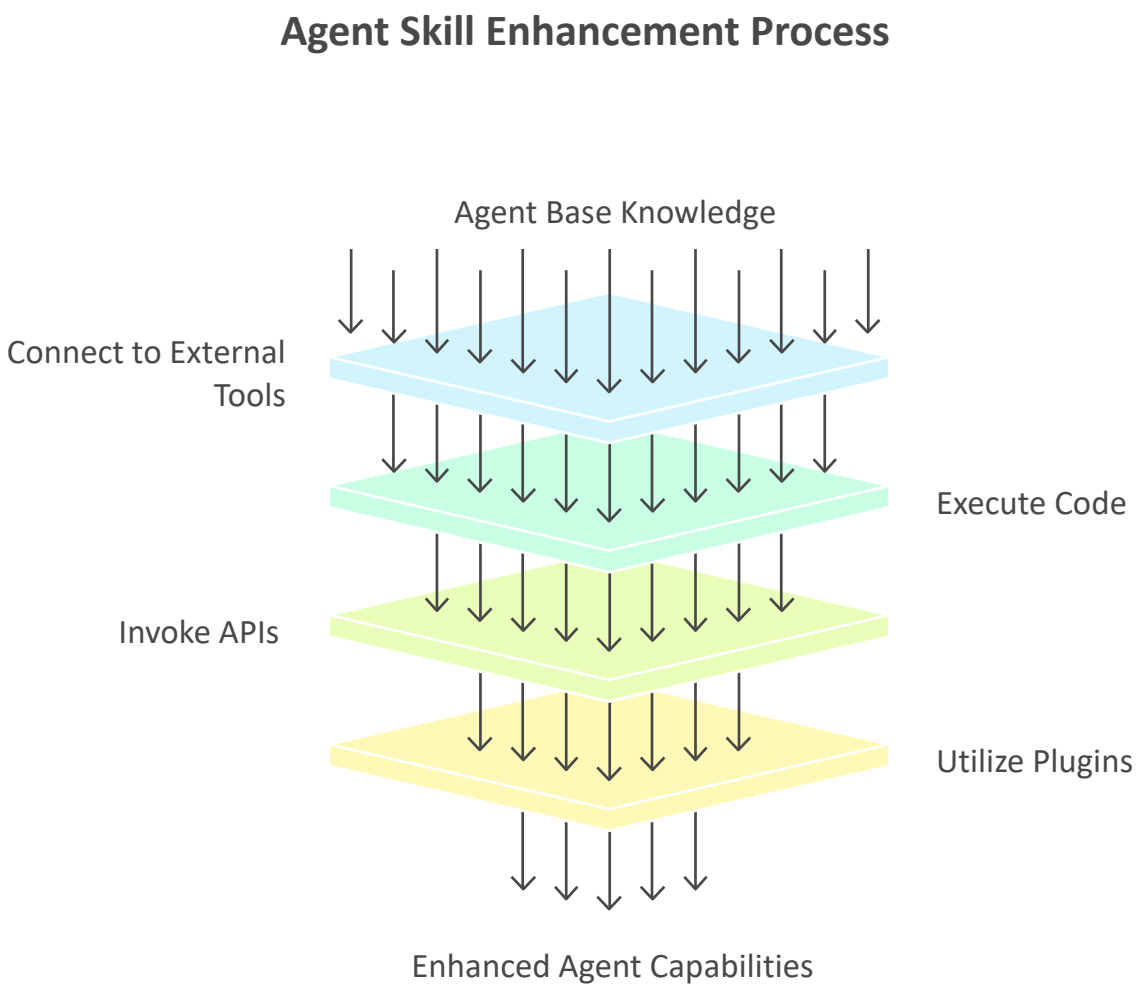
In practice, *Azure Functions* are used here to implement the tool protocol: each tool is an Azure Function endpoint that expects a standardized payload. This abstracts tools behind a consistent API boundary – fulfilling the TAP/AGP concept in the agentic model. For agent-to-agent, if using **gRPC**, a protobuf interface might define services for common agent actions, and all agents implement that. If using REST/HTTP, Azure API Management can enforce that all internal agent APIs share a common header or schema. **Azure Logic Apps or Durable Functions** also participate in protocol realization by handling error events or compensation flows when an agent raises an exception condition. For example, if an agent emits an "EthicalConcern" event, a Logic App could catch it and route to a compliance officer or trigger another agent to handle it, according to protocol rules.

NeuroStack's protocol layer is also forward-looking: it can integrate emerging standards like OAP. Microsoft's OAP (Open Agent Protocol) is intended to allow agents from different vendors to work together. By adopting OAP and MCP, NeuroStack ensures agents can call external services and other agents in a standardized way, future-proofing the platform. Notably, MCP support means agents can use thousands of community-provided tools via a simple protocol. In fact, Portkey's gateway and CrewAI Enterprise support MCP to let LLMs call remote tools. NeuroStack's gateway (Layer 2/3) will similarly mediate function-calling: an LLM's function call request (per OpenAI's format) can be converted to an internal tool API call through this layer.

Variations: If Kong Gateway is used, the *protocol enforcement* can be done via plugins – e.g. Kong can transform incoming agent messages to the required format or validate JSON schema. Portkey's advantage is native MCP support: Portkey can directly handle an LLM's request to call an MCP tool by forwarding it to the right server. In an enterprise with multiple AI systems, NeuroStack's Protocol layer can act as a translation layer between different agent frameworks. For instance, if one agent speaks OAP and another uses a proprietary format, the platform could provide a protocol adapter (possibly as an Azure Function or microservice) to translate between them.

Overall, the Protocol layer guarantees **standardization and traceability** in agent interactions. All messages and calls are logged (e.g. via Application Insights) with rich metadata, so one can audit who did what and when – crucial for compliance in regulated industries. This layer sets the rules of engagement that keep multi-agent workflows coherent and governable.

Layer 4: Tooling & Enrichment Layer (External Tools & Data Access)



Role & Logic: The Tooling & Enrichment layer equips agents with the ability to extend beyond their base knowledge by connecting to external tools, data sources, and services. This is essentially where *Retrieval-Augmented Generation (RAG)*, API calling, code execution, and other specialized skills come into play. Through this layer, an agent can fetch needed information, execute code, invoke external APIs (web services, databases), utilize plugins (e.g. CRM lookup, weather service), or perform computations – effectively giving the agent "hands and eyes" in the wider digital world. It's a critical layer for enterprise AI because it allows integration with existing systems and data.

Logical Architecture: Logically, each agent can be thought of as having a toolbox of **capabilities** attached. This layer manages those capabilities. For example, an agent may have a **Web Search tool** for internet queries, a **Database Query tool** to fetch enterprise data, a **Calculator or Code Exec tool** for computations, or a **Custom API tool** (like "SubmitOrder" calling an ERP system). When the agent's reasoning (Layer 5) decides it needs outside info or action, it invokes one of these tools. A key part of this layer is **retrieval of knowledge**: the platform might implement RAG by letting an agent query a vector database or Cognitive Search index to get relevant documents, which the agent then uses to formulate answers. Another part is *enrichment services* like text-to-speech, vision recognition, etc., which can be considered tools as well.

The design principle is modularity – new tools can be added over time without changing the agent's core logic. Each tool is defined with a name, description, and input/output schema, often registered in a central tool registry. In CrewAI, tools are typically Python functions marked with **@tool**, meaning they can be called by name from an agent prompt. NeuroStack likely extends this by hosting tools as services. For instance, if an agent has a "MedicalDatabaseLookup" tool, logically there is a defined interface for that: e.g. *Tool*: **lookup_patient_record(patient_i** -> returns patient data. The agent's prompt can trigg that action through the orchestrator.

The enrichment part refers to enhancing the data: e.g. summarizing retrieved text, extracting structured info, etc. But often this is handled by the agent's LLM itself. So primarily this layer is about hooking into **external data and APIs**.

Deployment Architecture: In Azure, implementing this layer involves a combination of managed services for each tool type and ensuring secure connectivity. For retrieval, **Azure Cognitive Search** (with semantic vector search) is a centerpiece – it serves as the enterprise knowledge index that agents query for grounding data. For example, when an agent needs up-to-date info from company documents, it uses an Azure Cognitive Search index loaded with those documents. The query can be formulated by the agent or via a query rewrite helper, then the top relevant snippets are returned to the agent (through the orchestrator) to incorporate into its reasoning. This essentially provides the agent with **long-term knowledge** beyond its static training data.

For external tools, **Azure Functions** or **Logic Apps** are used to implement them as microservices. As noted in Layer 3, each tool is wrapped in a uniform interface (likely an HTTP-triggered Azure Function) so that an agent's tool call goes out as an HTTP request to that function. Azure Functions provide scalability and isolation – each tool can scale independently and failures in one won't crash the agent process. Example: A *"ScheduleAppointment"* tool calls a hospital's scheduling API; it's implemented as an Azure Function that the agent triggers with appropriate params. Another *"GenerateReport"* tool might call Microsoft 365 API to create a document; again through a function or Logic App. The **Azure API Management** gateway can also publish certain internal APIs as tools – e.g. if the company has an internal service (say a *PricingService*), APIM could expose it to the agent environment with proper policy, effectively acting as a tool endpoint.

Some tools might require secure data access: e.g. querying an Azure SQL Database or Cosmos DB for specific info. In those cases, the agent might call an internal API (function) that has the necessary credentials to query the database and return results. **Azure Data Factory** or **Synapse** could be invoked for heavy data prep tasks (though likely asynchronous). In NeuroStack, an agent can even trigger pipelines via tools – e.g. calling a Data Factory pipeline to gather and preprocess data, then proceeding once the data is ready.

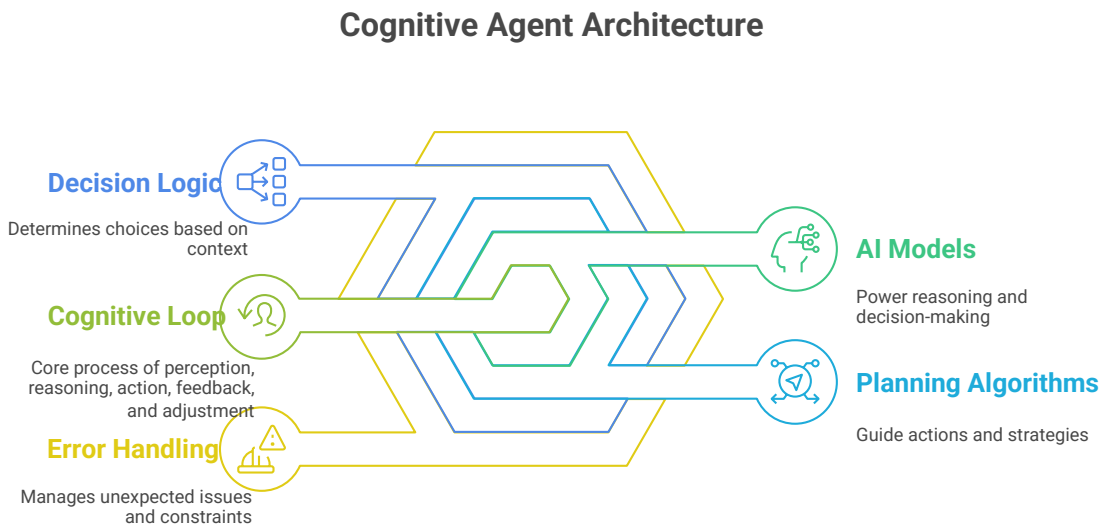
Because the platform encourages modular tools, adding a tool is as simple as deploying a new Azure Function and registering it. CrewAI's design makes it easy to assign tools to agents in their definition code. This means a developer or even a no-code *Agent Studio* interface can bind "Tool X" to "Agent Y" without much fuss. The **Tool registry** could live in a Cosmos DB or config file that lists available tools, their endpoints, and which roles are allowed to use them.

Governance: Importantly, this layer is subject to governance by Layer 8 – every tool integration must be monitored and controlled. For example, if a tool returns sensitive data (PHI from a medical API), the system tags it and ensures it isn't stored in logs or prompts beyond the session. Azure's compliance features help here (e.g. Azure Healthcare APIs are HIPAA-compliant, Cognitive Search can enforce ACLs on index data). The ops layer monitors tool usage to catch misuse or unusual patterns. We log each tool invocation (via App Insights or a central log) for audit trails.

Variations: Using **Kong Gateway** as part of tooling – Kong can manage external API endpoints and present them internally. For instance, each external API could be a Kong route, and the agent's tool call could just hit Kong with a specific path. Kong's plugins can even handle authentication for those external APIs. Kong also has an **AI Enrichment plugin** for RAG that can fetch knowledge base info for an LLM, though not as specialized as Cognitive Search. **Portkey AI Gateway** natively supports integrating tools via MCP – meaning if an agent (LLM) says it wants to use a tool, Portkey can route that request to a configured function or plugin. Portkey's approach is to allow developers to register tools that the LLM can call through the gateway as function calls. In NeuroStack, however, much of this is handled within the agent orchestration rather than the raw gateway.

In summary, Layer 4 provides agents the ability to *act on the world*: retrieve info and execute actions. By using Azure Functions, Cognitive Search, and secure APIs, NeuroStack ensures agents can reach enterprise data silos and services in a controlled manner. Agents effectively get "hands and eyes" to augment the base LLM's knowledge, which is crucial for solving real business tasks.

Layer 5: Cognition & Reasoning Layer (Agent Brain and Decision-Making)



Role & Logic: The Cognition & Reasoning layer is the "brain" of each agent – where it **understands context, plans actions, makes decisions, and learns from feedback**. This layer encompasses the AI models (especially LLMs) that power reasoning, as well as the planning algorithms, decision logic, and error handling strategies that turn raw model outputs into goal-directed behavior. In essence, this is what differentiates an autonomous *agent* from a simple chatbot: the agent can set subgoals, devise multi-step plans, call tools, adjust its approach based on results, handle unexpected errors, and even incorporate ethical or feedback constraints. Layer 5 is responsible for the agent's *cognitive loop*: perceive -> reason -> act -> get feedback -> adjust.

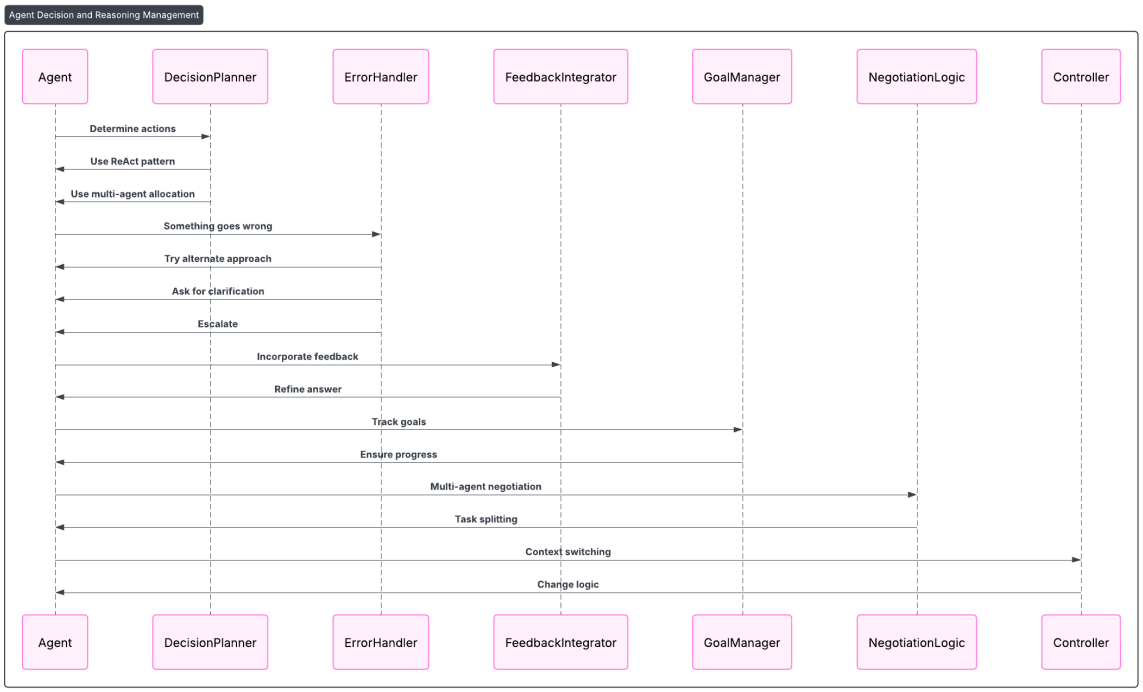
Logical Architecture: At the core of this layer is usually a **Large Language Model (LLM)** or a set of models (which could include logic engines or domain-specific models). The agent uses the LLM to interpret instructions and generate outputs. Surrounding the LLM, we have a **planner/orchestrator** that can break high-level goals into tasks. For instance, CrewAI employs a "Crew" orchestration where a *Chief Agent* can delegate tasks to specialist agents. The reasoning layer for a single agent might use techniques like **chain- of-thought prompting**, **tree-of-thought or self-reflection**, or other strategies to improve reasoning quality. Agents maintain an internal **state** (short-term memory, recently used tools, etc.) to inform their next action.

Key components logically: - **Decision Planner:** Figures out *what* sequence of actions or sub-tasks to perform to achieve the agent's goal (could be as simple as the ReAct pattern: decide to either answer or use a tool at each step, or as complex as multi-agent task allocation). - **Error Handler:** If something goes wrong (tool fails, no useful info found), the reasoning layer decides whether to try an alternate approach, ask for clarification, or escalate. - **Feedback Integrator:** The agent can incorporate feedback – e.g. if a reward model or human feedback indicates the answer was poor, the agent can refine it. - **Goal Manager:** In a continuous agent, this keeps track of the overarching goal and completion criteria. For multi-turn agents, it ensures the agent is making progress and not looping.

In multi-agent configurations, this layer might also involve **negotiation logic** (two agents discussing how to split tasks) or a *controller* that switches contexts between agents. But within one agent, it's mainly the *LLM and surrounding logic*.

Decision Planner: Figures out *what* sequence of actions or sub-tasks to perform to achieve the agent's goal (could be as simple as the ReAct pattern: decide to either answer or use a tool at each step, or as complex as multi-agent task allocation). - **Error Handler:** If something goes wrong (tool fails, no useful info found), the reasoning layer decides whether to try an alternate approach, ask for clarification, or escalate. - **Feedback Integrator:** The agent can incorporate feedback – e.g. if a reward model or human feedback indicates the answer was poor, the agent can refine it. - **Goal Manager:** In a continuous agent, this keeps track of the overarching goal and completion criteria. For multi-turn agents, it ensures the agent is making progress and not looping.

In multi-agent configurations, this layer might also involve **negotiation logic** (two agents discussing how to split tasks) or a *controller* that switches contexts between agents. But within one agent, it's mainly the *LLM and surrounding logic*.

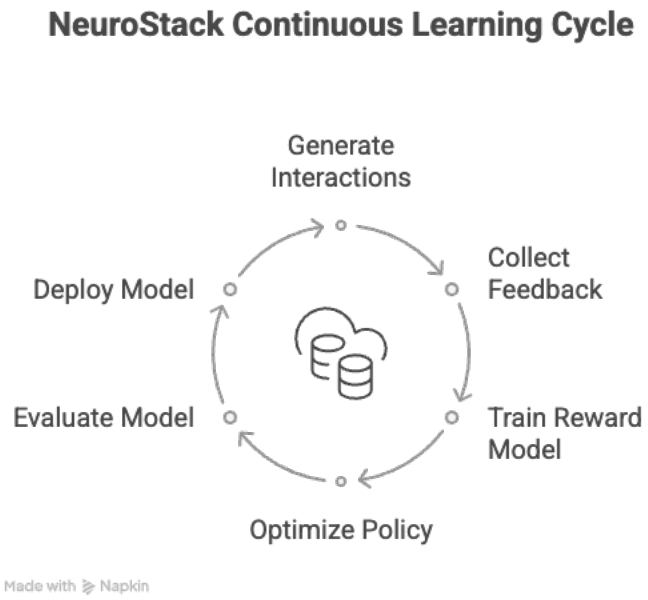


Azure Implementation (Models): NeuroStack leverages Azure's AI services to power this layer. The primary engine is **Azure OpenAI Service** – providing enterprise-grade access to models like GPT-4 and GPT-3.5. Each agent is configured with an appropriate model (or set of models): for example, a complex reasoning agent might use GPT-4 for its stronger capabilities, whereas a simple support agent uses GPT-3.5-turbo to save cost. CrewAI allows specifying different LLMs per agent role, and NeuroStack adopts this flexibility. Azure OpenAI ensures data privacy (no training on your data, and it's HIPAA-eligible for healthcare scenarios) – a crucial point for enterprises.

Beyond the base LLM, *other cognitive components* might supplement reasoning: e.g. **Azure Cognitive Services** for language understanding or custom ML models. For instance, an agent might use a smaller **Azure ML custom model** to do a specific classification or prediction as part of its reasoning pipeline. Azure ML can host a reinforcement learning policy or a rules-based decision module if needed.

Continuous Learning & RL: A key enhancement for layer 5 is *continuous improvement via reinforcement learning*. NeuroStack implements continuous learning through both **Reinforcement Learning from**

Human Feedback (RLHF) and autonomous feedback loops: - During operation, agents generate a lot of interactions. NeuroStack can log these and use them to fine-tune agent behavior. For example, if users consistently rate the agent's responses or if certain outcomes are desired (like successful task completion), those signals form a **reward signal**. - **Reward Models:** The platform trains reward models that score an agent's output quality or alignment. Using Azure Machine Learning, data scientists can label some agent



outputs as good or bad, train a reward model (often a smaller transformer that predicts a reward score), and register it in an Azure ML registry. NeuroStack supports versioning these reward models and deploying them (via Azure ML endpoints or containerized services). - **Reinforcement Learning Pipelines:** Azure ML's pipeline capability is used to orchestrate RL training runs. For example, a pipeline could periodically take the latest batch of interactions, compute rewards using the current reward model, and perform a **policy optimization** (like Proximal Policy Optimization – PPO) on the agent's LLM (or a smaller fine-tunable model) to improve it. The pipeline might use open-source libraries like **Hugging Face TRL (Transformer Reinforcement Learning)** or **Ray RLlib** to implement the RL algorithm at scale. Ray RLlib is useful to leverage multiple GPU machines to fine-tune models in parallel, and it can run on Azure ML clusters. Hugging Face's TRL and datasets can be integrated to perform RLHF; e.g. using a pairwise comparison approach to refine the LLM with human feedback. - After each training iteration, the newly fine-tuned model (or updated policy) is evaluated and if it shows improvement, it's versioned (e.g. "AgentModel v1.2") and deployed to the platform (perhaps as a new Azure OpenAI deployment or a custom model endpoint). This continuous learning loop allows **agents to improve their performance over time** – learning from their successes and mistakes.

For example, NeuroStack might integrate with **Azure Personalizer** (which is an RL-based personalization service) for certain decisions – Azure Personalizer can learn user preferences by reinforcement learning in an online fashion . In a support chatbot scenario, Personalizer could choose the style or tone of the response that yields the best user engagement, giving a form of continuous improvement in user satisfaction.

Real-Time Adaptation: The reasoning layer can also adapt per session using short-term feedback. For instance, the platform might employ an **LLM-as-a-Judge** pattern (an AI model that reviews the primary LLM's output against criteria) as a guardrail. If the judge model (or some heuristic) finds the answer lacking, the agent can loop to refine its answer before presenting to the user. This is a form of on-the-fly reinforcement (though not long-term training, it's immediate adjustment).

Error Handling: Using Azure Durable Functions or an orchestrator, if an agent encounters an error (e.g. tool failure), the reasoning logic might trigger a compensating action (retry with different parameters, call an alternative agent, or escalate to a human). This is implemented as part of the workflow definition for the agent – e.g. a Durable Functions orchestrator could catch exceptions and decide next steps, or the CrewAI framework's internal logic handles it via task status checks.

In summary, the Cognition & Reasoning layer is where the heavy AI thinking happens. NeuroStack uses top-tier models (GPT-4, etc.) and enhances them with planning logic and continuous learning. Over time, agents become smarter through RL. The layer is designed to adapt and improve based on outcomes, completing the feedback loop of agentic AI where agents not only act but also learn from their actions.

15

Layer 6: Memory & Personalization Layer (Short/Long-term Memory, User Context)

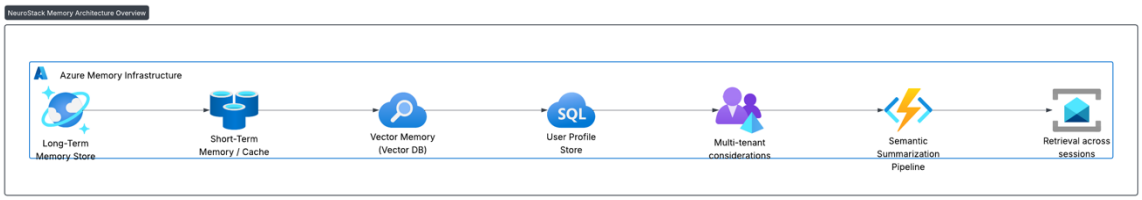
Role & Logic: The Memory & Personalization layer manages what the agents remember and how they tailor behavior to specific users or contexts. This covers both *short-term memory* (working memory of the current conversation or task) and *long-term memory* (knowledge accumulated across sessions), as well as user-specific data like preferences, profiles, and history. Essentially, this layer gives agents continuity and a sense of "self" or "user" context – enabling an agent to recall past interactions, and adapt responses to a user's needs or an organization's context.

A guiding principle here is to provide **"unlimited" memory** to agents beyond the fixed context window of an LLM. Since LLMs can't natively retain information across sessions, NeuroStack's memory system externalizes the memory into databases and uses retrieval to bring relevant pieces into the context when needed. Personalization ensures that if User A and User B interact, the agent can respond differently (appropriately) based on their profiles and past interactions.

Logical Architecture: Logically, this layer can be seen as consisting of: - **Working Memory Store:** A place to keep recent dialogue or task state (for quick recall in ongoing conversations). This might be ephemeral and limited (like a rolling conversation buffer). - **Long-Term Knowledge Store:** A persistent repository of facts the agent has learned or could reference later. For example, after a long analysis, the agent might store the outcome summary so it can recall it next time without recomputation. - **Vector Memory (Semantic Memory):** A vector database of embedded texts from past conversations or documents. This allows semantic search to retrieve relevant bits by meaning, not just exact words. When a new query comes, the agent finds semantically similar past contexts to include. - **User Profiles & Preferences:** Data about users (or any entity the agent personalizes for). This could include a user's name, role, settings, past queries, etc., as well as organizational policies. - **Multi-Session Linking:** Mechanisms to link sessions by the same user or on the same case. For instance, a *conversation ID* or *user ID* keys into stored memory, so the agent can fetch "everything we discussed last time."

Memory Operations: When a conversation with an agent ends, the content can be distilled (summarized) and stored as long-term memory. NeuroStack likely uses **semantic summarization** to periodically compress interaction history so that it can retain essential facts indefinitely without storing verbatim transcripts of unlimited length (helpful for privacy and storage). Summaries can be embedded as vectors for future retrieval.

Personalization logic: Before responding, an agent can fetch the user's profile (e.g. their account info in banking, or their medical history in healthcare) from this layer to ground the answer in known info about the user. This makes responses more relevant and customized. Preferences (like preferred language, formality, any domain-specific settings) are also applied.



Deployment Architecture: NeuroStack uses a combination of **Cosmos DB, Azure Redis Cache, Azure Cognitive Search, and vector databases** to implement memory.

- **Long-Term Memory Store:** Azure Cosmos DB is used to store long-term conversational data and

user-specific info in a flexible JSON format. Cosmos is ideal as it's globally distributed (if needed) and can store items like chat logs, summaries, agent learning artifacts, etc., with the ability to query them. For instance, every conversation turn might be an entry (with userID, timestamp, and text), or each concluded session results in a summary document stored in Cosmos. Cosmos DB can scale and handle fast reads/writes, and supports partitioning – NeuroStack likely partitions data by tenant or user id, ensuring multi-tenant isolation in a SaaS scenario. Key fields might be encrypted if containing PII/PHI (Cosmos allows client-side encryption as needed).

- **Short-Term Memory / Cache:** Azure **Redis** is used for quick-access working memory. For example, if an agent needs to store a temporary variable or the last prompt during a reasoning chain, Redis provides a low-latency store. Redis could also serve as a session cache for ongoing conversations (with TTL set to session duration). Additionally, Redis can back the semantic cache for LLM calls (as done in Portkey or LiteLLM gateways) – storing recent prompt embeddings and responses to quickly retrieve if a similar question is asked again.

- **Vector Memory (Vector DB):** For semantic memory, there are options. Azure Cognitive Search now has vector search capability which can function as a vector store for long-term memory. NeuroStack uses **Azure Cognitive Search with vector indexing** as a semantic memory store. This means that all important text (documents, conversation summaries, etc.) are indexed and embedded. When needed, the agent (or orchestrator) will compute an embedding of the current query or context (likely using the same model used for indexing, e.g. OpenAI's text-embedding-ada) and ask Cognitive Search for the most similar items. This retrieves pieces of prior knowledge that can be injected into the LLM prompt. For example, if a user asks "What did we decide about project X last meeting?", the vector search can find the summary of that meeting in memory and supply it. Cognitive Search provides enterprise-grade features like security filters, so an agent for one client won't retrieve vector data belonging to another (enforcing multi-tenancy). Alternatively, **Azure OpenAI Embeddings + an external vector DB** (like Pinecone, Weaviate, or Qdrant) can be used. Indeed, CrewAI supports multiple vector DBs (Qdrant, Weaviate, Pinecone, etc.), and NeuroStack could allow plugging those in if desired. If using **Azure Cosmos DB** with its new vector indexing or **Azure SQL** with PGVector, those could also hold embeddings. The key is that semantic similarity search is available for memory recall.

- **User Profile Store:** User profiles might reside in **Azure Cosmos DB or Azure SQL** (depending on if a relational schema is useful). For example, a financial advisor agent might retrieve a client's portfolio data from a SQL database. Cosmos is used if flexibility or JSON storage is needed (it can store profile docs with varying attributes per user). The platform ensures each tenant's profiles are isolated (Cosmos can use a partition key for tenant, or separate databases for strict isolation). These profiles can be fetched via a personalization service or directly via an internal API.

- **Multi-tenant considerations:** In SaaS mode, memory data must be tenant-isolated. NeuroStack enforces this via Cosmos DB partitions and role-based access control on resources. In other words, each customer's data has a tenant ID tag; agents will include that context to only retrieve that tenant's memory. If needed for high-security clients, NeuroStack can even deploy separate instances of the memory stores per client (e.g. a dedicated Cosmos DB or Search index). This is part of deployment flexibility.

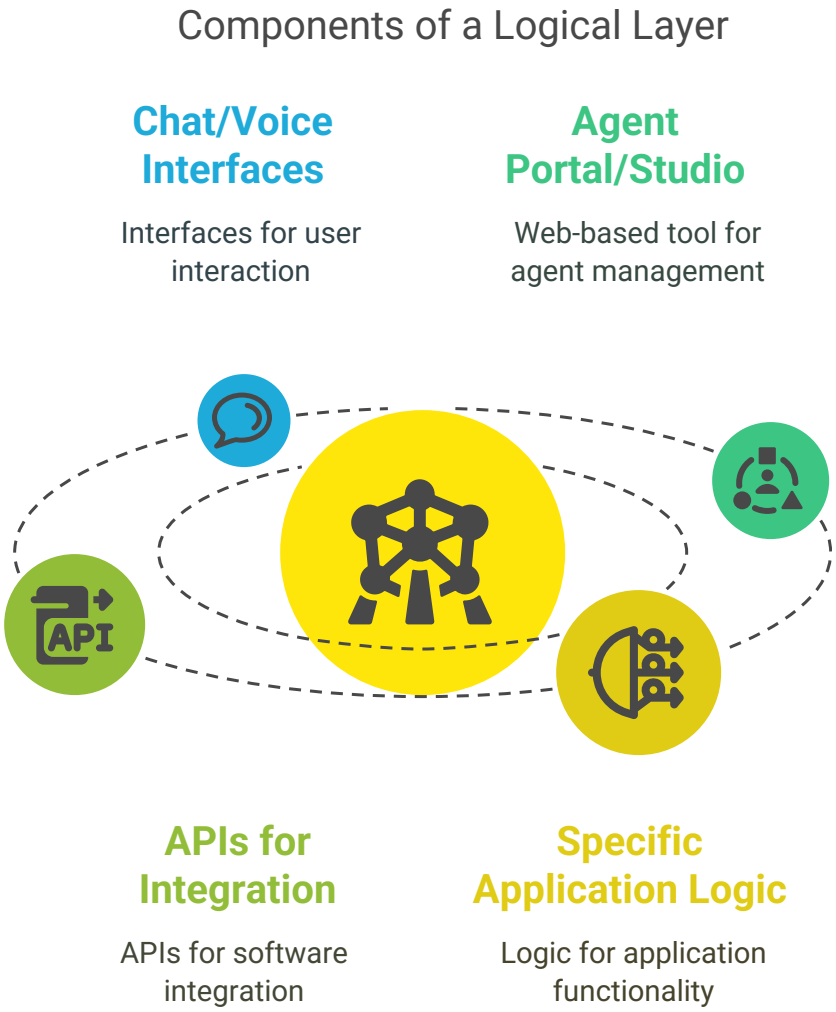
- **Semantic Summarization Pipeline:** Over time, long conversation logs could become huge. The system likely uses an automated summarization pipeline (could be an Azure Function or Logic App that triggers when a conv log exceeds a threshold). It would take the raw log, prompt an LLM to summarize it, store the summary (embedding it for search) and either archive or prune the raw log.

This gives “infinite memory” by summary chaining: older stuff is present in summary form.

- **Retrieval across sessions:** Suppose an agent has multiple sessions with the same user; the system will retrieve relevant info across all past sessions. This is achieved by vector search over all that user’s past conversation summaries or knowledge. For example, if a patient asks a health agent “what were my test results last time?”, the agent searches the vector memory of that patient’s records and prior conversations to answer accurately.
- **Personalization in practice:** Before generating a response, an agent can pull personalization data. For instance, an agent sees user = Alice (ID 123), it calls the profile DB (maybe via a tool) and finds Alice’s preferred name, her last few queries, and that she likes concise answers. The agent can then adjust tone or detail level accordingly. In Azure, one might incorporate **Azure Personalizer** or simply store preferences in Cosmos and have the agent logic adjust. Personalization rules can also be encoded in the system prompt (e.g. “This user’s preference: casual tone”).

Variations: Outside Azure, one could use any vector DB like Pinecone, or an open-source like Chroma for memory. The architecture is flexible: in fact, Greg Coquillo’s overview considered Pinecone/Weaviate part of the “Agent Internet” layer (network-accessible memory stores). The main point is an accessible, scalable memory that agents can query. Kong or APIM aren’t directly involved in memory, except that they could be used to secure access to memory APIs. For instance, an agent’s request to fetch from Cognitive Search could go through APIM which adds an OAuth token or enforces rate limits. But typically agents would call Azure Cognitive Search REST API or SDKs directly from their environment, using an API key or Managed Identity.

Summary: Layer 6 ensures that agents *remember relevant information and personalize their behavior*. Cosmos DB provides the durable memory of conversations and profiles. Azure Cognitive Search (or vector DB) provides the brain’s long-term semantic memory to recall facts across sessions. With these, NeuroStack agents can maintain context over indefinite interactions and tailor responses – e.g. remembering a customer’s previous issues in customer service, or a patient’s history in healthcare – thus making the interaction more effective and human-like over time.



Layer 7: Application Layer (User Interfaces & Applications)

Role & Logic: The Application layer comprises the **interfaces and applications through which end-users interact** with the agentic platform. It's essentially the front-end or integration point: chatbots, virtual assistants, web or mobile apps, or integrations into other systems (like an agent embedded in MS Teams, Slack, or a CRM). This layer also includes any *no-code/low-code interfaces* for building or configuring agents (for instance, an "Agent Studio" UI for admins to create agent workflows). In short, it's how humans (or external applications) see and use the intelligent agents.

Logical Architecture: On a logical level, this layer can include: - **Chat/Voice Interfaces:** e.g. a chat web app for support, a voice assistant interface (which would use STT and TTS). - **Agent Portal/Studio:** If provided, a web-based interface for creating or managing agents (drag-and-drop tool assignment, etc.). - **APIs for Integration:** The platform might expose certain capabilities via APIs so that other software can invoke agents. For example, a customer could integrate NeuroStack's agent into their own mobile app via a REST API call to the agent. - **Specific Application Logic:** If the agent is part of a larger app (like a workflow automation tool), that app logic is in this layer. For instance, an "AI Writing Assistant" application will have UI logic to collect user prompts, show draft outputs, allow edits, etc., with the agent providing suggestions under the hood.

NeuroStack, being general-purpose, can support many types of applications: personal assistants, content generation tools, research assistants, etc.. Each is an "application" that uses the core agent platform.

Deployment Architecture: In Azure, the application layer is implemented with services for web/mobile front-ends and integration: - **Web Apps and Services:** The primary UI front could be delivered via **Azure**

App Service or **Static Web Apps** for web UI, or a custom front-end deployed on a VM. This UI would connect to the agent back-end via API calls (likely through the APIM gateway). - **Bot Framework/Teams Integration:** If building chatbots, **Azure Bot Service** can be used to integrate with channels like Teams, Skype, Telegram, webchat, etc. The Bot Service can forward messages to an Azure Function or API (through APIM) that connects to the NeuroStack agent. For example, a Teams chat with a "Wealth Assistant" bot goes through Azure Bot Framework, which calls the NeuroStack API to get a response, and then replies to the user. - **Mobile Apps:** For mobile, one might embed the communication by calling the NeuroStack REST API from the app. Azure App Center could be used for mobile deployment, but that's more devops – the main link is the API. - **Agent Studio (No-code Builder):** If NeuroStack provides a UI for users to compose multi- agent workflows, that web interface might be an Angular/React app served via Azure App Service. It would allow selecting agent roles, tools, and deploying an agent. Under the hood, it might store configurations in Cosmos DB or Git (as mentioned, agent definitions could be saved in Cosmos or YAML files). - **Integration APIs:** APIM is likely used to expose certain endpoints externally. For instance, a **REST endpoint** for "Ask the Clinical Agent" could be exposed to integrate with an EHR system. External systems then call that API with necessary parameters. APIM handles authentication (Azure AD OAuth, etc.) and passes the request to the agent orchestrator. This allows enterprise apps to use the AI without directly dealing with model details.

- **Multi-Channel Output:** For voice, Azure Cognitive Services Speech can be used. The application layer would capture user speech (via an app or device), send it to **Azure Speech-to-Text**, get the transcribed text, feed it to the agent, then take the agent's text response and use **Text-to-Speech** to speak it out. This is how a voice assistant or call-center agent would work.

- **User Authentication & Identity:** The front-end likely uses enterprise SSO (Azure AD) so that the user's identity and roles are known. That identity is propagated to the back-end (e.g. via JWT in APIM). This is important so that the agent can personalize responses and also enforce permissions (some info might be visible to managers but not to others, etc.). The application layer ensures secure sign-on and possibly displays different UI or options based on user role, but we also use that identity in back-end calls for filtering memory or knowledge base (so an agent only accesses data the user is allowed to see).

To illustrate: consider a **Healthcare Clinical Assistant** application – it might be delivered as a tab in the hospital's EHR system (application layer integration). A doctor opens the "AI Assistant" tab, which loads a web app (maybe served via Azure App Service) that calls the agent's API. The doctor's login (Azure AD) token goes along, the agent uses it to query only that doctor's patient data. The UI shows a chat interface where the doctor asks, "Summarize this patient's last 3 visits." The agent fetches from memory (patient records via Cognitive Search) and responds in the UI. This entire flow crosses the application layer (UI and integration point) down to the reasoning and back.

Variations: Many enterprise clients will integrate the agent into their existing software. The architecture supports that by providing APIs and modular front-ends. For example, a bank could integrate the Wealth Assistant into their online banking site via API calls. Or a manufacturing company might put a Predictive Maintenance agent into their monitoring dashboard UI.

As for **Kong/Portkey**, these primarily play at the API gateway level (Layers 2/3), not directly at UI. However, Kong does offer a Developer Portal (for publishing APIs to external devs) which could be relevant if NeuroStack offers an API for third parties – APIM likewise has a Developer Portal for API consumers. Portkey doesn't have a full portal (it's focused on internal use). If NeuroStack were to expose a platform for

developers to build agents, that portal might be considered part of the application layer as well (since it's an interface for a different kind of user – a developer user).

In summary, Layer 7 is about delivering the agent's capabilities to end-users in convenient forms – whether through conversational UIs, integration in existing apps, or new apps entirely built around AI agents. It ensures a user-friendly, secure, and accessible interface to the powerful multi-agent system underneath.

21

Layer 8: Operations & Governance Layer (Ops, Monitoring, Compliance, Cost Control)

Role & Logic: The Operations & Governance layer focuses on **running the platform reliably and within guardrails**. It includes system monitoring, logging, compliance enforcement (security, privacy), budgeting and cost management (FinOps), and governance policies (like approving new agents or changes, auditing behavior). Essentially, while other layers enable the AI's functionality, Layer 8 ensures it is **safe, compliant, cost-effective, and well-managed** in production. This layer is crucial for enterprise trust – it's where we put the checks and oversight so that autonomous agents remain under control and accountable.

Logical Architecture: Key aspects of this layer include: - **Observability & Logging:** Comprehensive tracking of system metrics and agent behaviors. Every request, response, tool invocation, error, etc., is logged. Metrics include latency, success/failure rates, and critically, token usage (since token = cost in LLM world). The observability system might have real-time dashboards for these metrics. - **Audit Trail:** The platform records agent decisions and actions for later audit. For instance, in BFSI, if a compliance agent approves a trade, there must be a log of that decision (with timestamps, what data was considered, etc.). This trail is stored (possibly in an append-only database or Azure Log Analytics with retention) for compliance audits. - **Policy Enforcement:** The system can have rules like "Agents of type X are not allowed to use tool Y" or "The AI should never output certain sensitive info." These policies can be implemented via gateway filters (for content) or in the agent's logic as guardrails. Possibly a **Prompt Filtering** or content moderation check runs on outputs to redact PII or block disallowed content. If something violates, the ops layer could trigger an alert or a fallback (e.g. ask a human to review). - **Security & Access Control:** Ensuring only authorized users/agents access certain data (this ties into identity and the memory access controls, but from ops perspective, making sure secrets are secure, least privilege for tools, etc.). Also includes periodic security reviews of prompts to avoid prompt injection issues – some frameworks add **prompt firewalls** in this layer. - **Cost Management (FinOps):** Monitoring and controlling how many tokens or API calls are used, to avoid runaway costs. Possibly setting **quotas** per user or per agent per day. The ops layer can preemptively route expensive requests to cheaper models if cost budgets are exceeded (like degrade gracefully). It can also produce reports of usage by department for chargeback. - **Model and Version Management:** Governing when a new model or agent version is promoted to production. Possibly having an approval workflow or A/ B testing (canary releases) for new model versions (like testing GPT-4 vs GPT-4.1, etc.). - **Deployment & CI/ CD:** Overseeing the CI/CD pipelines for agent code, tool code, and infrastructure changes. Ensuring that updates go through testing and do not break compliance. Azure DevOps or GitHub Actions pipelines would be configured to deploy new agents/tools with approvals.

Deployment Architecture: Azure provides a lot of built-in capabilities here: - **Monitoring & Metrics:** Use **Azure Monitor and Application Insights** extensively. All APIs (if using APIM) automatically send metrics to Azure Monitor – APIM can emit custom metrics like token counts via policies. We enable **diagnostic logging** on APIM such that full request/response payloads (with appropriate masking for sensitive data) go to **Log Analytics**. We instrument the agent runtime (maybe via Application Insights SDK in the code) to log internal events too. These logs feed dashboards. Azure Monitor workbooks or Power BI can show live token usage,

response times, etc. as described. - **Central Logging:** Azure Log Analytics aggregates logs from APIM, Functions, AKS (container logs), etc. We set retention as needed (maybe 30-90 days or more for compliance). We can query these logs for auditing – e.g. show all actions by Agent "ComplianceAgent" on a certain day. - **Alerts:** Azure Monitor Alerts are set up for anomalies (e.g. sudden spike in errors or usage). If an agent starts failing or cost skyrockets, it pages DevOps. - **FinOps controls:** APIM and gateways have **quota policies**. In APIM, we can set call quotas per subscription, but for token-based quotas, we instrument metrics and possibly implement custom logic (like a policy that tracks token count header). Since APIM alone doesn't count tokens natively, we rely on our metrics: e.g. if our custom metric for monthly tokens > threshold, the ops team might get alerted or the gateway might start rejecting requests with a message (APIM can do conditional check with `quota-by-key` although token needs custom calc). Other gateways (Kong, Portkey) have built-in budget limit features. Portkey for instance allows setting a hard budget per API key and will stop requests beyond that. In an Azure-native approach, we could integrate with **Azure Cost Management** APIs or custom scripts to monitor spending on Azure OpenAI etc. - **Guardrails and Content Moderation:** Azure OpenAI has a content filtering mechanism for certain abuse content. We also integrate **Azure Content Safety** for more advanced moderation on prompts and responses. The ops layer can run every output through Azure Content Safety API – if it flags something (hate, sexual, etc.), we log it and either redact or adjust. For domain-specific guardrails (like HIPAA), we implement checks: e.g. if an agent tries to output PHI outside authorized context, the Ops layer should catch it. Perhaps we add special tokens or tags in prompts for sensitive data and a post-processor removes them if unauthorized. - **Tool Governance:** Each tool invocation is logged as well (with parameters sans sensitive info). Ops can analyze tool usage patterns (e.g. if an agent unexpectedly calls an admin tool, that's suspicious and can be flagged). Access to tools is role-based – only agents with permission can use certain tools, enforced by an internal ACL (maybe maintained in Cosmos DB or config, and checked in the orchestrator before dispatching a tool call). - **MCP / Function-calling governance:** If using an AI gateway like Portkey, it offers prompt security and guardrails framework (for injection, toxicity, etc.). Those can be configured to sanitize inputs or outputs. We might use such features if Portkey is in play, or mimic them via Azure policies.

- **Model Management:** Azure ML or Azure OpenAI handles model versioning. We might use **Azure OpenAI on your data** features for RAG as well, but focusing on architecture: The ops team would have a process to update the model (e.g. deploying GPT-4 2025 version). This may involve testing in a staging environment (maybe using APIM revision or a separate instance). Canary releases can be achieved with APIM or Portkey: e.g. Portkey supports **canary testing** configs to route a small percentage to a new model. APIM could route internal testers to a different backend deployment via a header or parameters. This is more procedural but important for governance to ensure quality.

- **Identity & Access Management:** The platform integrates with **Azure AD** for all user-facing components (as mentioned for Layer 7). From ops perspective, that ensures compliance with enterprise IAM (no anonymous access; all actions are tied to an identity). Also, internal service identities (Managed Identities) are used for components to access secrets or call other services. Secrets for tools (API keys, etc.) are stored in **Azure Key Vault**, and tools/agents fetch them via managed identity so that no plain secrets are in code.

- **Compliance Certifications:** By building on Azure, NeuroStack benefits from Azure's compliance (e.g. HIPAA, SOC2, GDPR compliance). Layer 8 ensures that when dealing with regulated data, proper measures are in place. For example, in healthcare, Azure's Health Data Services (FHIR API) and HITRUST certification help meet standards. Data at rest is encrypted (Azure manages that, plus we

can do app-level encryption for especially sensitive fields). We also design disaster recovery and backup strategies at this layer (so logs and data are backed up for compliance and continuity).

Variations (Kong, Portkey): - **Kong Enterprise** provides a **runtime dashboard** and can emit fine-grained metrics (latency, tokens via plugin) to monitoring systems. Kong also has plugins for **open policy agent (OPA)** or custom Lua scripts which could enforce governance rules at the gateway. - **Portkey** has a built-in monitoring UI that tracks ~50+ metrics per request including token counts, model chosen, etc.. It is basically made for observability and FinOps with budget monitoring and multi-tenant usage reports. If Portkey is used, we could leverage its UI for a lot of ops metrics out of the box, and integrate its logs with Azure Monitor if needed. - The provided comparison chart highlights differences: APIM has deep Azure integration for monitoring (pushes everything to Azure Monitor), whereas Portkey and others have their own but can export. For **cost tracking**, APIM by itself is partial (it doesn't know token costs), Kong tracks tokens and has dashboards, Portkey is first-class in FinOps with budgets, LiteLLM also auto-calculates spend per provider. So if cost control is a priority, Portkey or LiteLLM can complement APIM's functionality by providing detailed cost analytics that APIM lacks. In an Azure-native approach, we can achieve similar results with custom logging and scripts, as described, but it requires more engineering.

Summary: Layer 8 is about **trust and manageability**. It wraps around the entire stack to ensure the AI platform runs within set bounds and can be continuously monitored and improved. With this layer, enterprises get the confidence to deploy NeuroStack at scale: they can see exactly what agents are doing (full observability), control what they *are allowed* to do (policies, guardrails), ensure compliance and security, and manage costs and performance proactively. This layer makes the platform enterprise-grade by providing the oversight that matches the organization's risk and governance requirements.

With the 8-layer architecture defined, we now delve into specific critical components and use cases: the **AI Gateway** design (enterprise control point for AI calls), advanced **Cognition & Reasoning** via continuous learning, **Memory & Personalization** techniques, concrete industry **Use Case architectures**, and deployment **flexibility** across SaaS and private cloud.

2. AI Gateway Layer (Enterprise Control Point)

One of the core pieces of the NeuroStack platform is the **AI Gateway** – an enterprise control point through which AI requests flow. This gateway can be seen as spanning Layers 2 and 8 (communication and governance), as it intermediates between clients (Layer 7 apps) and the AI backend, and enforces policies (Layer 8) on those interactions. Its architecture and features are critical for security, reliability, and efficiency.

Gateway Purpose: The AI Gateway handles all incoming and outgoing agent interactions with LLMs and tools. It serves as the **traffic manager and policy enforcer**: every user query to an agent, and often every agent call to an LLM or external model, goes through the gateway. This allows central enforcement of rules and optimizations such as *policy enforcement, semantic caching, intelligent routing, failover, observability logging, authentication (JWT validation), rate limiting & quotas, function call mediation,* and *cost control*. We detail each of these and how they're implemented.

NeuroStack can implement this gateway using Azure's native API Management, or alternatives like Kong Konnect (Enterprise) or the open-source Portkey AI Gateway. Each has trade-offs, discussed after describing the core design.

2.1 Architecture and Features of the AI Gateway

Policy Enforcement: The gateway is configured with policies that govern usage. This includes **content policies** (e.g. block prompts that contain disallowed content or filter out sensitive info in responses) and **tool access policies** (e.g. an agent's function call is allowed only if the agent has permission). In Azure APIM, policy statements can be used to inspect requests and responses. For instance, APIM can use a custom policy to run the prompt through a content moderation model before sending to the LLM – if it returns high risk, APIM can reject the request or mask it. Similarly, if a response contains a forbidden phrase, APIM could strip it or replace it. The gateway ensures compliance with corporate policies like data cannot leave certain boundaries (if a request is asking for something outside allowed scope, it can be blocked). Another angle is **tool policy**: since all function/tool calls ideally pass via gateway, it can check "Is agent X allowed to call tool Y?" If not, it returns an error or asks for escalation. This centralizes control rather than relying on each agent's code alone.

Semantic Caching: To reduce latency and cost, the gateway employs caching of LLM responses. Unlike typical API caching which uses exact matches, here we use **semantic cache** that can recognize similar requests even if not identical. For example, if two users ask the same question wording differently, the gateway can detect similarity and reuse a prior answer. Implementation: When a request comes in, the gateway generates an embedding of the prompt (using a fast embedding model). It then checks a **cache store** (like Redis or a vector DB) for existing vectors that are above a similarity threshold. If a near-match is found, the cached response is returned immediately, saving a call to the LLM. If not, the request is processed by the LLM and the result is stored in the cache (with the embedding of the question) for future. The cache entries have a TTL (maybe a few hours to days, configurable). Semantic caching is extremely useful for repeated queries – Portkey's documentation notes up to ~20% cache hit rates with ~99% accuracy in some cases. In our platform, this could drastically cut token usage for recurring questions (common in Q&A style agents).

Technically, we can implement this by having a sidecar or policy: APIM alone doesn't do vector caching natively, but one approach is to integrate an **Azure Cognitive Search index** as a vector cache or use **Redis with vector similarity** (Redis with RediSearch supports vector similarity search). We might have an Azure Function that APIM calls in a policy: it sends the user question to that function, which computes embedding (using Azure OpenAI embedding model or a local model) and queries the vector store. If a result above

similarity threshold is found, the function returns the cached answer (with maybe a header indicating it's a cache hit) and APIM short-circuits to return that to client. If no hit, APIM continues to call the LLM and then we store that result via another call or background process. This is somewhat complex in APIM policy, but doable with the policy to an Azure Function.

In **Kong Gateway (Konnect)**, their AI Gateway offering has built-in semantic caching capabilities through plugins – likely leveraging a vector store and embedding model behind the scenes. Similarly, **Portkey** has semantic caching built-in and configurable via its API, using either in-memory or Redis + an embedding service. Portkey can ignore system prompts for caching (so that different system instructions don't prevent cache hits on similar user queries). We can cite that APIM would need custom integration whereas Kong/ Portkey have out-of-box solutions for this.

26

Mixture-of-Expert Routing (Dynamic Model Selection): The gateway can route requests to different backend AI models or agent instances based on the content or context of the request – essentially implementing a **mixture-of-experts** approach[4] . For example, we might have multiple LLMs: GPT-4 for complex queries, a smaller model or a domain-specific model for simple or specialized queries (like a code model for programming questions, an open-source model for questions that require on-prem processing, etc.). The gateway inspects the incoming request (it could look at metadata like an "intent" or just analyze keywords or required tools) and decides which model or agent is best suited.

In practice, one approach is to do **intent classification** at the gateway: e.g. if a request contains code or is from a developer context, route to a Code model; if it's a general question, route to GPT-4. Another approach is based on cost: if user is on a basic tier, use a cheaper model unless necessary. This can be configured in Portkey's conditional routing – e.g. *if user plan = free then model = GPT-3.5, if plan = premium or question is complex then model = GPT-4*. Portkey's conditional routing rules allow checking request parameters or metadata to choose a target model. Kong can achieve similar with its routing and plugin logic (possibly requiring an external classifier). APIM is more limited but one could use a policy that if prompt length > X or contains certain keywords, call a different backend.

NeuroStack's gateway uses a combination of *static rules and AI-based classification*. For example, we might have a lightweight classification model deployed (or use Azure AI Studio's custom classification) that labels an incoming query with a category. APIM could call an Azure Function that returns a label, then APIM chooses the backend based on that label (APIM policies can do conditional on context variables). This ensures the "expert model" handles relevant queries. We also could run multiple models in parallel for certain requests and have the gateway merge results (though that's advanced and costlier – probably not default but an option for critical queries needing consensus or ensemble).

The benefits are **improved quality and cost optimization**: use specialized models where they shine, and cheaper ones for routine tasks. For example, Developer queries go to a Codex-like model (or Azure GPT-4-32k with code), while simple FAQs go to a smaller model to save tokens. The gateway acts as a smart dispatcher in a pool of experts, abstracting this from the client.

Failover and Load Balancing: The gateway ensures high availability by load-balancing across multiple instances of backend services (e.g. multiple Azure OpenAI deployments, or multiple replicas of a local model) and automatically **failing over** if one is unavailable. For Azure OpenAI, an enterprise might have a primary instance (maybe with a set throughput) and a secondary (pay-as-you-go) as backup. The gateway can be configured to route primarily to the first, but if it hits capacity or errors, switch to second. APIM can use its built-in **load balancing** between a set of backend URLs – we can list two backends with a certain weighting or priority. APIM doesn't automatically know to failover on 429 though, so we'd implement a policy: if response status = 429 or 503, retry on the other backend (APIM has retry policies and can use conditional call to alternate backend). Indeed, APIM with a combination of *priority-based backends* and *circuit-breaker logic* can achieve this: e.g. treat one instance as primary until it fails, then shift traffic. A *Retry- After* header can be respected by APIM by waiting or by temporarily not using that instance.

Kong and Portkey have this natively. Kong can do upstream load-balancing with health checks and automatic failover. Portkey explicitly mentions automatic retries and failovers if one provider fails. It can distribute calls across multiple API keys or endpoints to maximize throughput and avoid hitting rate limits. This way, if one model endpoint returns error or is slow, the gateway shifts to another, and clients see uninterrupted service (maybe a slight delay on retry, but handled gracefully).

We also consider *geo-routing*: If NeuroStack is deployed across regions, the gateway might route user requests to the nearest region's AI backend for latency, and failover to another region if one goes down. APIM can do region-based routing using location headers or separate regional gateways under a global Front Door.

Observability & Logging: The gateway logs detailed metrics for every request. This includes: timestamp, requesting user/app, which agent or model was called, prompt size (token count), response size, latency, any error codes, etc. Especially important, it logs **tokens used** (both prompt and completion). In APIM, we achieve this by capturing the `content-length` of requests and responses and mapping that (for Azure OpenAI, the service returns usage data in headers that we can log). If using Azure OpenAI, the response includes headers like `opai-processing-ms` and `opai-usage-prompt-tokens` etc. We can have APIM

or a backend function log those. If using Portkey or Kong, they natively measure tokens and cost per request. These metrics are forwarded to monitoring dashboards in real-time.

We set up **Application Insights** or custom telemetry in APIM to push metrics like token count as custom metrics. APIM policies allow an `emit-metric` step – we could emit "PromptTokens" and "CompletionTokens" metrics to Azure Monitor, using values parsed from the response body or headers. Over thousands of requests, these metrics feed into dashboards to track usage by day, by agent, etc. Logging also captures sample prompts and outputs for debugging (though possibly with sensitive info masked). As described earlier, these logs and metrics give *real-time visibility* into how the system is performing and where costs are going. For example, an ops engineer could see that one particular client is driving 50% of token usage, or that a certain type of query often misses cache and has high latency, prompting optimization.

JWT Validation & Identity: The gateway sits at the boundary, so it's the ideal place to enforce authentication. We require all requests carry a valid **JWT (JSON Web Token)** from Azure AD (or another IdP in enterprise). APIM can validate JWTs using built-in policy (verify signature, audience, etc.) and even enforce scopes/roles. Kong has plugins for JWT and can integrate with OpenID Connect for Azure AD. Portkey's OSS gateway by default uses API keys, but enterprise mode supports JWT and SSO integration. Regardless of implementation, NeuroStack's gateway will reject any request that is not properly authenticated or authorized. The JWT's claims (like user ID, roles, tenant) are then used for personalization and policy decisions downstream (as noted in Layers 6 and 7).

The gateway can also map users to subscription plans using JWT claims or an internal lookup. APIM's **Product subscriptions** can issue subscription keys; we might not expose that publicly if we rely on JWT from SSO, but for external developer usage of the API, subscription keys or JWT from a service principal might be used. In any case, identity is verified and then propagated – the gateway might pass the user ID to the agent logic via a header or as part of the payload, so the agent can know who is asking.

Token Quotas & Rate Limiting: To prevent abuse and control costs, the gateway enforces quotas on requests. Azure APIM supports rate-limit policies (X calls per minute) and total quota policies (N calls per day/week per subscription). We adapt this to the AI context. We might limit each user to e.g. 1000 messages per month, or more granularly, to $ worth of tokens (which is harder to do in APIM directly). For approximation, we could convert tokens to an estimated number of calls, but better is to use the observed token metrics. With the metrics logging in place, we can accumulate usage per user. If using APIM alone, an external script or function might periodically disable a subscription that exceeds budget (APIM has APIs to

modify subscription state). Or APIM could consult an external service on each call (less ideal performance-wise).

Kong and Portkey handle this more directly: Kong's plugin can rate-limit by tokens (Kong's AI gateway tracks token usage per consumer). Portkey allows *budget limits* by cost or token count built-in. LiteLLM also supports quotas per user/time window. In a multi-tenant SaaS, we'd assign each client or user a quota which the gateway enforces to ensure no one racks up unlimited bills.

Even within one organization, quotas protect against infinite loops or runaway agents. For example, an agent might accidentally get into a long reasoning loop consuming thousands of tokens – the gateway can notice one request exceeding a threshold and cut it off (OpenAI API itself has max tokens per request, but if an agent keeps calling recursively, a gateway-level circuit breaker can detect unusual patterns and intervene).

MCP Function-Calling Mediation: Modern LLMs (like OpenAI's GPT-4) can perform **function calling**, where the model returns a structured request to call a tool. The AI Gateway can act as the broker for these function calls. Specifically, Portkey's gateway directly supports MCP – an LLM can output a function call (with function name and JSON arguments), and Portkey will intercept that and call the corresponding tool plugin or remote MCP server, then feed the result back to the LLM. In NeuroStack, if we implement function calling without Portkey, we could still do it: APIM or the orchestrator could detect a function-call response from the model and then dispatch to the appropriate tool (this likely happens in the agent logic rather than APIM itself). However, if using **OpenAI function calling**, the API can be invoked such that the model directly returns the function call and the client (our orchestrator) must then execute the function and call the model again with the result. The gateway could facilitate this by having an orchestrator component or using Portkey's ability.

Given the architecture, we likely integrate **MCP (Model Context Protocol)** to have a library of tools accessible via network. The gateway helps by centralizing configuration of those tools (like tool endpoints and auth keys) and by routing the LLM's call appropriately. If using Kong, Kong can simply host the tool endpoints as routes – the agent code still has to handle the LLM's function call output, but it can call the tool via Kong (which might add security and monitoring on that call as well). Portkey would do a lot automatically (the LLM's output can directly trigger a tool call defined in Portkey config). So the AI Gateway ensures that **function calling is done securely and efficiently**: tools are executed on sanctioned endpoints, with audit logging. It may also sanitize the function inputs (to avoid prompt injection via function arguments).

FinOps Cost Control: We've touched on this with quotas and caching, but in broader terms, the gateway acts as a central point for **cost visibility and control**. By logging all usage and tagging it, the enterprise can attribute costs to teams or features. The gateway can enforce **budget limits**: e.g. if a department has a monthly budget for AI calls, the gateway monitors and once near limit, it could either alert or start routing those calls to a cheaper model or a simulated response. Possibly, the gateway might even degrade functionality – e.g. no long answers or no GPT-4 usage after budget cap (the routing logic can handle that). With the detailed metrics recorded (token, model, user), finance teams can do chargeback easily.

Portkey and LiteLLM both highlight cost analytics: Portkey users often cut AI costs by 30-50% due to these optimizations (caching, routing), and it provides multi-tenant cost reports and even can set *hard budget cutoffs*. In Azure APIM, such features would be custom-built as described, but achievable. The important

point: the **gateway optimizes runtime costs** via caching and routing, and gives full insight to optimize prompts and usage policy over time.

To summarize, **NeuroStack's AI Gateway** is a sophisticated API gateway tailored for AI workloads, incorporating: - Content and usage policies for **safety**, - **Semantic cache** and *dynamic model routing* for **performance and cost**[4], - **Load balancing/failover** for **reliability**, - **Auth and quotas** for **security and cost control**, - **Observability** for **monitoring and optimization**, - **Function-call handling** for **tool use orchestration**.

All of this works transparently: a user query comes in, the gateway might hit the cache (or else pick a model), send to the model, maybe handle function calls, return the answer, and log everything – all in a way that the user just sees a good answer faster and the business sees lower costs and high control.

2.2 Implementation Variations: Azure APIM vs Kong vs Portkey

Azure API Management (APIM): Chosen when deep Azure integration and full API lifecycle management are priorities. APIM's strengths: - Native Azure AD integration for auth, Managed Identity for calling Azure backends. - Seamless logging to Azure Monitor (turnkey observability in Azure). - Developer portal, versioning, tier management out-of-box (useful if exposing services to third-party developers). - Microsoft- managed security and compliance – APIM meets enterprise security standards and is maintained by MS (less ops burden).

However, APIM lacks AI-specific features like semantic cache or content-based routing out-of-box. We compensate via custom policies and Azure Functions, but that adds complexity. Also, APIM has throughput limits – we must ensure scaling (premium tier for VNET + multi-region support if needed). If all components are in Azure, APIM often is the safe enterprise choice because it's "battle-tested" and aligns with enterprise processes (monitoring, compliance).

Kong Gateway (Konnect or self-hosted): Kong is a high-performance API gateway with flexible plugin architecture. Using Kong (possibly Kong Enterprise with Konnect control plane) might be ideal if the solution must run hybrid or on-prem (since Kong can run anywhere), or if certain AI gateway plugins provided by Kong make integration easier. Kong's strengths: - Very **extensible** via plugins (Lua or WASM plugins). There are community or enterprise plugins for rate limiting by token, for logging, etc., and one can write custom logic to call external AI services for classification or to implement semantic caching. - Kong Enterprise offers a **UI/dashboard** for monitoring and a developer portal, though not as tightly integrated as APIM's with Azure's ecosystem. - It can do dynamic routing and has an API for config, but might still need some custom code to e.g. analyze prompt content (though Kong's plugin could call an ML model for that). - Kong's **AI Gateway edition** (if available via Kong Labs) specifically touts features like token-based rate limiting and AI observability, and guardrail policies (like prompt filtering for PII), which indicates Kong is recognizing those needs.

Trade-offs: Kong requires managing the infrastructure (if self-hosted), including scaling and patching. Konnect (their cloud) can manage control plane but data plane still needs hosting. Also, some advanced features might be in Kong Enterprise (licensed). In contrast, APIM is fully managed by Azure. If the client is Azure-centric, APIM might integrate more easily with their systems (e.g. using Azure AD, Key Vault, etc. natively). If they want cloud-agnostic or on-prem, Kong might win.

Portkey AI Gateway: Portkey is a specialized open-source AI gateway designed exactly for LLM orchestration. Key advantages: - **AI-specific features built-in:** semantic caching, conditional routing, multi- LLM support, streaming support, function calling (MCP), and **50+ metrics** per request tracking (including hallucination flags, token counts, etc.). - **Observability:** It has an integrated monitoring UI showing those metrics in real time. Great for debugging and optimizing prompt usage. - **Guardrails:** Portkey offers an AI guardrails framework (like prompt injection protection, custom guardrails via small models). This can save effort building those from scratch. - **Ease of multi-backend integration:** it supports over 200+ models/ providers via adapters (OpenAI, Azure, local like HuggingFace, etc.), so adding a new model is often just config. This is useful if NeuroStack might incorporate models from different vendors (Azure OpenAI, AWS Bedrock, local GPU, etc.). APIM or Kong can connect to any HTTP endpoint too, but Portkey has done the work to integrate with these APIs nicely (e.g. unified interface). - **Cost-centric features:** Portkey explicitly supports budget limits, cost tracking per key, etc. out-of-box.

Trade-offs for Portkey: - It's a newer open-source project; while rapidly developing, it might not have the enterprise maturity (in terms of support, rigorous security vetting) as APIM or Kong. For an enterprise like 's clients, using an OS project might require careful review. - It might not have full features of a general API gateway – e.g. a developer portal or advanced user management (though it has teams and org separation concept for internal use). - Running Portkey gateway means self-hosting (likely in a container on AKS). It's lightweight (~120KB binary) and high-performance by design, so that's a plus for scaling edge deployments. - But if a customer is already invested in APIM or Kong, introducing Portkey is an additional component to maintain, unless it fully replaces some of their API gateways for AI traffic.

Hybrid Approach: It's possible to combine them: e.g. use APIM for outer gateway (client-facing, doing JWT auth, quota) and Portkey underneath as an internal smart router for LLM calls. For instance, APIM could receive a request, do auth and basic rate-limit, then forward to Portkey which does model selection, caching, etc. However, that introduces complexity and latency overhead. More straightforward is choosing one robust solution.

Recommendation Considerations: - If the client strongly prefers Azure-native and needs tight integration with Azure services and compliance, **APIM** is likely recommended. APIM's enterprise features (dev portal, lifecycle, security assurances) are valuable, and the team can implement the AI-specific parts via the described custom approaches. Given 's partnership with Microsoft (as indicated by their Agentic AI services focus on hyperscalers including Azure), APIM might be the default choice. The referenced solution design indeed chose APIM as the gateway citing reasons: Azure integration, enterprise features, and security/compliance benefits. - If the use case requires rapid innovation and using many model types (or if the clients are okay with an OSS component), **Portkey** could accelerate development with its built-in features. Portkey could be layered behind APIM (APIM as secure front, Portkey as the brain of LLM routing), or used stand-alone for internal traffic. For a purely on-prem deployment where Azure services might not be fully available, Portkey could shine by handling multi-model orchestration in a self-contained way. - **Kong** is a good middle-ground if the organization already uses Kong for APIs or if they want multi-cloud support with a proven API gateway. Kong can be extended to cover much of the needed functionality and might come with enterprise support which Portkey lacks. With Kong's plugin system and perhaps Kong's specific AI gateway capabilities (if using Kong's **API7/Apache APISIX** variant or similar, which also has AI routing plugins), one can implement semantic caching (using a vector store plugin) and dynamic routing (some examples from APISIX show conditional routing and multi-LLM LB capabilities).

Trade-off Summary:

- *APIM:* +First-class Azure integration (security, monitoring, compliance), +rich API mgmt features, +fully managed; – lacks AI-specific features (custom dev needed), – throughput cost (APIM can be expensive at very high call volumes).
- *Kong:* +High performance, +extensible with existing ecosystem, +on-prem friendly; – not specialized for AI (some features in enterprise plugins but not as turnkey as Portkey), – you manage infra (unless using Konnect cloud for control plane).
- *Portkey:* +Designed for AI tasks (caching, guardrails, multi-LLM etc. all in one) [4], +fast and low-resource, +open-source (no license cost); – new project (potential support risk), – fewer traditional API management features.

In many enterprise scenarios, **a combination** might be used: APIM at the edge for client-facing management, and Portkey internally for optimizing calls to models. But to keep architecture simpler, one might pick one. The provided design rationale in user files leans toward APIM for a production Azure-based solution, emphasizing its enterprise readiness even if alternatives have some more AI bells and whistles.

The AI Gateway is implemented with a focus on enterprise controls. If using Azure APIM, we configure custom policies for caching, routing, etc., leveraging Azure Functions and vector stores to supplement APIM's capabilities. If using Kong or Portkey, we utilize their built-in plugins and features to achieve similar outcomes. In all cases, the result is a robust gateway that **optimizes performance (through caching and routing) , ensures reliability (through failover), enforces security (through auth and policies), and provides full observability and cost governance (through metrics and quotas)** for the NeuroStack platform.

3. Cognition & Reasoning Enhancements (Continuous Learning with RL)

Layer 5 (Cognition & Reasoning) is the "brain" of the agents, and a major focus is enabling **continuous learning and improvement** of agent behavior. We will detail how NeuroStack implements reinforcement learning (RL) to achieve this, integrating with Azure Machine Learning and open-source RLHF (Reinforcement Learning from Human Feedback) frameworks. We also cover how **reward models** are trained, versioned, and deployed in the platform to close the feedback loop.

3.1 Continuous Learning via Reinforcement Learning

Goal: Allow agents to improve their performance over time by learning from experience and feedback, rather than being static. Traditional LLM deployments are fixed after fine-tuning; NeuroStack introduces a feedback mechanism so agents can be refined (either offline through training updates, or online through adaptive algorithms like Azure Personalizer for real-time tweaks).

Approach: We employ *Reinforcement Learning (RL)*, in particular **RLHF** where possible, as well as *Reinforcement Learning from AI Feedback* (RLAIF) or environment feedback. The general loop: 1. The agent produces outputs (answers, decisions). 2. A **reward signal** is generated evaluating those outputs. This could come from explicit human ratings (e.g. thumb up/down in the UI), implicit user behavior (did the user follow the recommendation or request a clarification?), or a heuristic/automated judge (an AI model scoring the output for correctness or style). 3. Using these rewards, the agent (or rather the model behind it) is optimized to increase expected reward on future outputs. This typically means adjusting model parameters via RL algorithms like **Proximal Policy Optimization (PPO)** or similar. 4. New policy (model) versions are tested and deployed, yielding gradually better performance aligned with human preferences or business goals.

Integrating Azure Machine Learning (Azure ML) RL pipelines: Azure ML provides a robust environment for orchestrating ML training, including RL. We design a pipeline in Azure ML that automates the RLHF process: - **Data Collection Step:** Continuously collect interactions. Logs of agent conversations are stored (in Cosmos DB and Log Analytics). We filter out a training dataset: e.g. (state, action, reward) tuples. In language terms, state could be the conversation context, action is the agent's response, reward is the score from human or model. - **Reward Modeling Step:** If using a learned reward model, we update it here. Azure ML can train a reward model on collected human preference data. For instance, if we have pairs of responses (one good, one bad) from human ranking, we fine-tune a smaller transformer that takes a response and predicts a scalar reward. We might use Hugging Face's `trlX` library or custom PyTorch for this. Many open-source reward models exist (OpenAI's default models, or ones like OpenAssistant's reward model) that we can use as starting points. - **RL Policy Optimization Step:** Use Azure ML to run PPO (or another algorithm) on the agent's policy (which is essentially the LLM or a subset of it if we use a smaller model proxy). Azure ML's distributed training capabilities (using multiple GPUs) help scale this. We might use **Ray RLlib** on Azure ML – RLlib can be launched in a training cluster to parallelize experience simulation and policy updates. The training uses the reward model from previous step to score outputs and adjust the policy accordingly. - **Evaluation Step:** Evaluate the new model on a validation set of prompts (including perhaps real user questions and some edge cases). Ensure it's improving the intended metrics (helpfulness, accuracy) without regressing on constraints (no policy violations). This can involve both automatic metrics and possibly a quick human review for critical use cases. - **Deployment Step:** If the model passes evaluation, we register the new version in **Azure ML Model Registry** (or deploy directly to Azure OpenAI if using fine-tuning there, though currently OpenAI doesn't support RLHF fine-tune directly by customers, so likely we manage our own

model or use OpenAI's in future if they allow custom policy hooking). - **Canary Release:** Deploy the new model perhaps as a separate deployment endpoint and route a small % of traffic via the AI Gateway (as described with conditional routing) to test in production safely. If results are good, gradually increase traffic and eventually switch over fully.

Using Azure ML's pipeline, we can schedule this process (e.g. nightly or weekly, depending on data volume). Also, Azure ML provides tracking (via logs, metrics in Azure ML Studio) for the training process, so we have experiment records for each model version.

Open-Source RLHF integration: We incorporate tools like: - **Hugging Face's Transformers Reinforcement Learning (TRL)** library – e.g. use `TRL` to do PPO training on top of a pretrained model. HuggingFace has examples for training a model with a reward model (like training GPT-2 to respond with certain style using a BERT-based reward model). We can adapt those methods for our larger language models, possibly using DeepSpeed or LoRA to fine-tune efficiently due to model size. - **DeepSpeed-Chat** or Microsoft's frameworks – Microsoft has released some recipes for RLHF on Azure (DeepSpeed library can do RLHF at scale on Azure infrastructure, leveraging GPU clusters). We could use those to handle large-scale models if needed (though fine-tuning GPT-4 is not feasible directly; likely we apply RLHF on smaller instruct models if we host any, or on open-source models). - **Ray RLlib** – focusing on the algorithm implementation and scaling. RLlib can be particularly useful if we consider more complex environment simulation (multi-agent RL, etc.). For example, if an agent can take a sequence of actions (like a reasoning chain), RLlib could simulate episodes of agent reasoning and optimize a policy for when to use tools, etc.

Continuous vs Batch Learning: Likely we do batch updates (offline RL training on collected data) periodically. However, some aspects can be online: - Use **Azure Personalizer** (which is basically an online RL bandit service) to fine-tune certain decisions in real-time. Azure Personalizer could be integrated for, say, selecting one of several styles or suggestions to present to a user. The "context" (user profile, current query) is sent to Personalizer with a few possible agent responses or action choices, Personalizer chooses one based on learned policy and gets reward from user behavior (like click or rating)[3]. Over time it personalizes the agent's decisions to the user. This is a form of continuous learning implemented as a managed service, complementary to RLHF which is more about aligning the model's general behavior. - We also might do **self-play or self-critique** where two agent instances debate or check each other's answers and that generates a reward signal (like forcing consistency). That is experimental but can be part of continuous improvement (like an agent and a "critic agent").

3.2 Training and Deployment of Reward Models

Reward Models: A reward model is typically a smaller neural network that scores an AI output given some context, representing how "good" that output is (according to human preferences or task-specific criteria). Training a reward model often involves collecting human preference data: show a human two outputs for the same input and ask which is better; use those comparisons to train the model to assign higher score to the better one (often via a Bradley-Terry or pairwise loss). In NeuroStack: - For general helpfulness/ harmlessness, we might leverage existing reward models (like OpenAI's or ones from OpenAssistant). But for domain-specific nuances (e.g. "medical correctness" or "adherence to company policy"), we likely need to fine-tune or train a reward model on data in that domain. - Azure ML is used to train these models as part of the RL pipeline. We store the labeled data (could be in Azure Storage or Azure Data Lake, and registered as Azure ML Dataset). We then run a training script (PyTorch) on a compute cluster with the base model (maybe a pretrained BERT or a smaller transformer) as a starting point. After training, we register the reward model in the Azure ML Model Registry.

Versioning: Each time we train a reward model or policy model, it's versioned (with metadata like date, data used, metrics achieved). We probably tag models with semantic versioning or experiment IDs. This is important because if a new version performs worse unexpectedly, we need to rollback to a prior model. Azure ML's registry or even Azure OpenAI's model deployment names can handle versioning (e.g. deploy new model under "AgentModel_v2" while "AgentModel_v1" still exists).

Deployment of Reward Models: How do reward models get used in production? A reward model might be used in two ways: 1. **Offline in training:** The reward model doesn't run during user queries, only in the background RL training loop to evaluate candidate outputs. In that case, it doesn't need a high-availability deployment; it can run in the context of the training job on Azure ML. You just ensure the latest reward model file is accessible to the training code. 2. **Online for Real-time Optimization:** Possibly to do on-the-fly reranking or self-critique. For example, an agent might generate multiple drafts of an answer and use a reward model to pick the best one before replying. If we want that, the reward model could be deployed as an **Azure ML online endpoint** or an internal service that the agent (or gateway) can call with a few candidate outputs and get scores. However, doing this for each query could add latency and cost. It may be reserved for important cases or done asynchronously as a suggestion. Another scenario is we use a reward model to gauge if an answer is good enough or needs a refinement iteration (like "did the agent follow the user's instructions fully?" if not, have it attempt self-correction) – such evaluation could use either a heuristic or a reward model acting as a "critic" agent (LLM-as-judge approach we mentioned).

In most RLHF setups, the reward model is primarily a training artifact, not used in production serving directly. We likely follow that: reward model is used in Azure ML training loops to produce new policy versions. We ensure the reward model is aligned with evolving user expectations by periodically retraining it as more human feedback comes.

Improving Agent Behavior with RLHF: By iteratively applying RLHF, agents should: - Produce outputs that are more **helpful, correct, and aligned** with user needs (since human feedback pushes in that direction). - Reduce unwanted behaviors (like refusals when not needed, or inaccuracies) because the reward can penalize those. For instance, if users mark an answer as incorrect, the reward model will learn signals that correlate with correctness and push the model to be more accurate next time. - Tailor style to what users prefer (some might prefer concise answers, others detailed; if the reward or personalization captures user satisfaction, the agent will adapt style accordingly).

Open-source RLHF frameworks use-case: For example, if we incorporate **Hugging Face's TRLX**: We could fine-tune an open-source model like LLaMA-2 on our domain using supervised fine-tuning first, then use TRLX to do PPO with a reward model. TRLX provides a straightforward API to do this by wrapping the model and reward. We could run that on Azure ML with multiple GPUs for efficiency. And since TRLX is open, we can inspect and adjust it to our needs (like customizing the reward function easily).

Another example is using **Ray Tune/RLlib** – if we want hyperparameter tuning or parallel rollout workers. Azure ML can run a Ray cluster; RLlib has multi-node support. That way, we can simulate many prompts in parallel (even using the agent to generate responses to some synthetic prompts to augment training) and gather reward feedback.

Human in the Loop: RLHF implies human involvement. We would integrate an interface in the Application layer (or as an admin tool) where human annotators or even end-users can provide feedback on answers. For instance, after getting an answer, a user might click 👍 or 👎. Those signals get stored and later used in training. Or internal domain experts periodically review agent outputs in critical tasks (like a compliance officer

reviews some trade approvals done by the agent and marks issues). This labeled data is invaluable for the reward model training. NeuroStack likely provides a feedback API or logging of these user ratings directly, building a dataset over time. We might also prompt users for specific feedback ("Was this answer useful?" prompt in UI).

Summation: NeuroStack's continuous learning uses Azure ML to orchestrate RLHF pipelines, supported by open-source frameworks for the heavy-lifting. Reward models are central to turning qualitative human preferences into a quantitative reward signal, and these are managed like any ML model (train, register, deploy as needed). Over time, this approach **improves agent decision-making quality** in a safe manner, because we always validate models before fully deploying them, and we incorporate human oversight where needed. This ensures agents remain aligned with user expectations and organizational values, even as those evolve – fulfilling the promise that agentic AI systems "learn and continuously improve autonomously" with proper guardrails.

4. Memory & Personalization (Layer 6) in Depth

Layer 6 has the crucial job of giving agents memory and customizing their behavior to individual users and contexts. We've outlined the architecture of memory in section 1, now we will elaborate specifically on how NeuroStack delivers **"unlimited" long-term memory** and fine-grained **personalization** using Azure services like Cosmos DB, Redis, Cognitive Search, and vector databases, including multi-tenant considerations and cross-session continuity.

4.1 Unlimited Long-Term Memory

Challenge: LLMs have context window limits (e.g. 4k or 32k tokens) and do not persist information across sessions. We need the agent to remember potentially vast amounts of info (user data, past conversations, world knowledge updates) beyond those limits.

Solution: Implement an **externalized memory store** that can grow arbitrarily, combined with techniques to summarize and retrieve relevant pieces on the fly: - Use **Vector Databases / Indexes** to store embeddings of all content that might be relevant later. This includes transcripts of all past interactions (summarized or chunked), documents ingested, outcomes of tasks, etc. As more data comes in, we add more vectors. This can scale to millions of items, far beyond any single prompt capacity. When needed, we query by embedding to pull out only the top relevant items. - Use **hierarchical summarization**: as conversation logs grow, periodically summarize older parts. The summaries themselves can be summarized further as needed (a summary of summaries), providing a way to compress potentially years of dialogue into a manageable state while preserving important details. The agent can thus recall essential facts without carrying the entire detailed history. - By chaining retrieval and summarization, we can approximate "infinite memory". For instance, if a user references something from a year ago, the system can search the vector store for that reference and surface a summary of that conversation to the agent. Even if that conversation was 100k tokens originally, it might have been distilled into a 500-token summary vector, which we can feed into the current prompt. - Use **knowledge base grounding**: For domain knowledge, integrate external knowledge sources into memory. Cognitive Search can index not only past dialogues but also relevant documents (manuals, FAQs, intranet pages). The vector search unifies these sources, so memory isn't just what the agent said before, but can include corporate knowledge that the agent should "remember" to use.

In practice, let's illustrate: Suppose an agent is an **IT helpdesk assistant** and a user comes back after a week saying "The solution you gave last time didn't work, what else can I try?" The agent didn't literally "remember" that conversation (since it's a new session), but Layer 6 steps in: it uses the user identity or conversation reference to search the **long-term memory store**. The vector DB finds the summary of last week's conversation (because the embedding of "solution you gave last time" is similar to content in that summary). The agent retrieves that summary: e.g. "Summary of conversation on Oct 1: User reported X issue, agent suggested solution Y." Now the agent has context to respond helpfully – it can avoid repeating the same solution and move to the next approach. From the user's perspective, the agent "remembered" the past talk, even though it was actually pulled from the database.

Cosmos DB for structured long-term memory: In addition to vector stores, we use **Cosmos DB** to store structured memory: e.g. full conversation logs and key-value memory. For instance, after a session, we might store in Cosmos an object:

```
{ userId: "Alice", sessionId: "12345", date: "2025-10-05",
  transcript: [ {...message1...}, {...message2...}, ... ],
  summary: "In this session, Alice asked about her account upgrade. Agent
explained options A and B." }
```

We might also have collections for e.g. “facts learned”. If during a conversation the agent had to look up some info (say a product price), it could store that in a “memory” collection as a fact (with tags, timestamps). This is akin to how humans make notes. Those facts can then be re-indexed to vector DB for retrieval.

Cosmos can store quite large data (but large text we often index in Cognitive Search for retrieval). We likely use Cosmos as the primary store for conversation records (for compliance, auditing) and possibly as a source for feeding into the search index. We might run an Azure Function triggered on new Cosmos items to push the text to Cognitive Search index (or vector index) for semantic search usage.

Redis for short-term memory and session continuity: During an active session, the agent’s state (like recent turns not yet summarized) can be kept in **Azure Cache for Redis**. Redis is very fast, and we can use it to store, for example, the last N user utterances and agent responses, accessible by a session ID key. This prevents us from having to re-send the entire conversation in each request across stateless boundaries – the agent orchestrator can fetch recent context from Redis and feed it to the model. Redis might also store ephemeral state like “has the agent already used tool X in this session?” which could influence logic (to avoid redundant calls).

Redis also can act as a **feature store** for personalization: quick flags like “userPref_summarize=True” or last seen time.

Azure Cognitive Search for multi-modal memory: We configure **Cognitive Search** with: - An index of conversation summaries (with fields like user, agent, date, content, vector embedding). - Possibly a separate index for knowledge documents (with their own vectors) that serve as long-term knowledge memory. - We use *hybrid search* (vector + keyword) to ensure precise facts can also be filtered (e.g. user asks specifically “order 123 status from last time” we might filter by user or order id in addition to semantic match). - Cognitive Search’s security trimming ensures that if by any chance indexes hold multiple tenants’ data, an API call with a user/tenant context only returns that tenant’s data. Alternatively, we maintain separate indexes per tenant for absolute separation. - It’s noted that Azure Search can function as a **vector database** for long-term memory, which we utilize.

Multi-tenant Vector Memory Stores: In a SaaS scenario, we have to ensure each client’s data is isolated in the memory layer: - **Separate indexes/collections per tenant:** The simplest approach – each tenant gets its own Cognitive Search index and Cosmos container. This guarantees no cross-talk. Downside is managing many indexes if we have hundreds of tenants, but Cognitive Search can handle multiple indexes (with limits). - Or use **tenantID as a filter**: We can add a field `tenantId` in each document and always query with a filter `tenantId:<X>` along with the vector search. Cognitive Search supports vector search with filter (the filter is applied post similarity search by default, but given small candidate sets that’s fine). This approach is used if we want one big index for all to leverage combined knowledge but logically we wouldn’t want that across companies, so likely separate indexes to be safe (especially for compliance). - Similarly, Cosmos DB can use the partition key as tenantId to isolate data at the DB level. - The system also may allow

some *global memory* (common knowledge) accessible to all, but any sensitive or private memory stays tenant-specific.

Semantic Summarization and Retention: We implement automated summarization: - After each session, an Azure Function uses an LLM to summarize the conversation. The summary is stored (in Cosmos and vector index). We might keep the full detail for X days in Cosmos for audit, but for quick search after months, we rely on the summary (or an even more compressed form). - If a conversation is extremely long, we might chunk it and summarize chunk by chunk then summarize those summaries (multi-level). - Summaries themselves are periodically reviewed. If a user returns to a topic, agent can combine new info with older summary and produce an updated summary (to avoid summary from months ago missing some new nuance). - The retention policy might be: raw logs older than say 90 days are archived (maybe to cold storage or deleted for privacy, if not needed), but their distilled knowledge remains in the long-term memory store indefinitely (since the summary or extracted facts are not as sensitive or heavy).

Retrieval Across Sessions: The memory system ensures an agent can retrieve info from any past session or related context: - Key is **indexing by semantic content** rather than session ID. So even if user doesn't explicitly reference a date or ID, the system finds the relevant info by meaning. - Example: Agent helped user with "Project Phoenix" in a session last month. Now user asks: "Give me an update on Phoenix's status." The word Phoenix triggers a vector search and finds that previous conversation summary or a document about Project Phoenix. The agent uses that to respond with continuity ("Last time, we had completed phase 1 of Project Phoenix. As of now..."). Without memory, the agent wouldn't connect the dots.

User Personalization Data: Beyond memory of conversations, personalization includes user preferences (like language, formality), profile info (role, expertise level), etc. This info likely stored in Cosmos DB or Azure AD (for some profile attributes). NeuroStack's memory layer accesses this via either direct DB query or through integration: - If integrated with enterprise systems, the agent could call a tool to get user profile (e.g. "getUserPreferences(userId)") which fetches from Cosmos or another DB. But to save that step each time, the orchestrator might pre-fetch basic profile at session start and provide it in the system prompt ("The user is a VIP customer who prefers brief answers."). - Personalization memory also means tracking **user's individual knowledge**. E.g. if an employee has had training, the agent shouldn't repeat basic info they already know. The agent might store "user is proficient in product X" in profile after noticing that. Next time, it might skip beginner explanations. This is an advanced dynamic, but the architecture supports it: just store these flags in memory per user, update them as needed. - Multi-turn personalization: e.g. if a user always asks for sources, the agent can learn that and always include sources. Could implement by storing a preference once user toggles a setting ("always provide sources: true").

Memory Access Optimization: We ensure that retrieving memory doesn't slow down responses noticeably: - Cognitive Search queries are fast (typically <200ms for vector search on moderate size indexes). We might restrict to searching top N past items to keep latency low. - We do memory fetch in parallel with other operations if possible (though usually, you need memory context before calling the LLM). - We cache recent memory lookups in Redis too – e.g. if within the same session the agent already pulled certain info, keep it handy. - The gateway or orchestrator might include memory fetch as part of its pipeline for each request: first fetch relevant memory, then construct the LLM prompt with both user query and memory, then query LLM.

Cognitive Search vs Vector DB vs Redis for Vector: We mention all. In Azure, Cognitive Search with vectors is appealing as a managed service and integrated security. Some might opt for a dedicated vector

DB like **Pinecone** or **Weaviate** especially if already in use; those could be hosted in Azure as well. If Pinecone/Weaviate is used, it's part of the "Agent Internet" – accessible via API. In Greg's layers, he put Weaviate/Pinecone in Layer 2 (Agent Internet) possibly because they act as an external network resource storing knowledge agents share. In NeuroStack, we can plug those easily since LangChain/CrewAI support them. But using Azure Cognitive Search reduces external dependencies and leverages Azure's enterprise features (role-based access, compliance).

Case: Multi-session conversation continuity in **Healthcare** (to foreshadow use case): A patient asks an agent, "How have my blood sugar readings been this past month compared to before?". The agent can retrieve from memory the summary of their readings or their last appointment notes. Perhaps those notes were indexed in vector DB. It finds "Sep: average 130 mg/dL, Oct: average 145 mg/dL" in a note. The agent then can answer trends. Without memory, the agent couldn't because it doesn't have patient data built in.

Scalability: As memory grows, we ensure our vector index scales. Azure Cognitive Search can scale partitions and handle millions of vectors (with some performance considerations). We will monitor index size and perhaps archive older stuff that's truly irrelevant (maybe using time-based filters if needed). But conceptually, with summarization, the bulk of "knowledge" remains manageable.

Data privacy & Compliance in Memory: Memory may contain personal or sensitive data (especially in healthcare or finance). We adhere to compliance: - Use Cosmos DB features like row-level security or encryption for certain fields if needed. - Data in Cognitive Search indexes is encrypted at rest and we can restrict who can query them (only the agent via managed identity, not arbitrary users). - Possibly de-identify data before putting into general vector store if needed for R&D (but if it's to help the same user, we often keep it identified). - Honor data retention policies (e.g. GDPR right-to-be-forgotten: if a user requests deletion of their data, we purge their memory records from Cosmos and Search). - Because we tag data by sensitivity, the ops layer (Layer 8) monitors that sensitive data is not stored longer or used in prompts beyond allowed session unless properly handled (like PHI tagging example). For instance, if PHI is fetched via a FHIR API, the system might not log it or might discard it after use, only keeping an abstract.

In conclusion, NeuroStack's Memory & Personalization layer creates a **continuously learning knowledge base for each agent and user**. It **blends databases and search indexes** to achieve a memory that is as vast as needed, yet accessible in split-seconds with relevant pieces. It also **personalizes each interaction** by recalling user-specific information (preferences, past interactions), giving the feel of a consistent, tailored experience across time. This layer is what enables agents to have context across sessions and to behave like long-term assistants rather than stateless chatbots.

1 How CrewAI is evolving beyond orchestration to create the most powerful Agentic AI platform
https://blog.crewai.com/how-crewai-is-evolving-beyond-orchestration-to-create-the-most-powerful-agentic-ai-platform/

2 Remote MCP - Portkey Docs
https://portkey.ai/docs/product/ai-gateway/remote-mcp

3 Enterprise Architecture for CrewAI-based Healthcare & BFSI SaaS Platform.pdf
file://file-JfwWfTRKyefDg3VYpTcdRr

4 Why Portkey is the AI gateway your team needs
https://portkey.ai/blog/the-right-ai-gateway-for-you

41